

神经网络

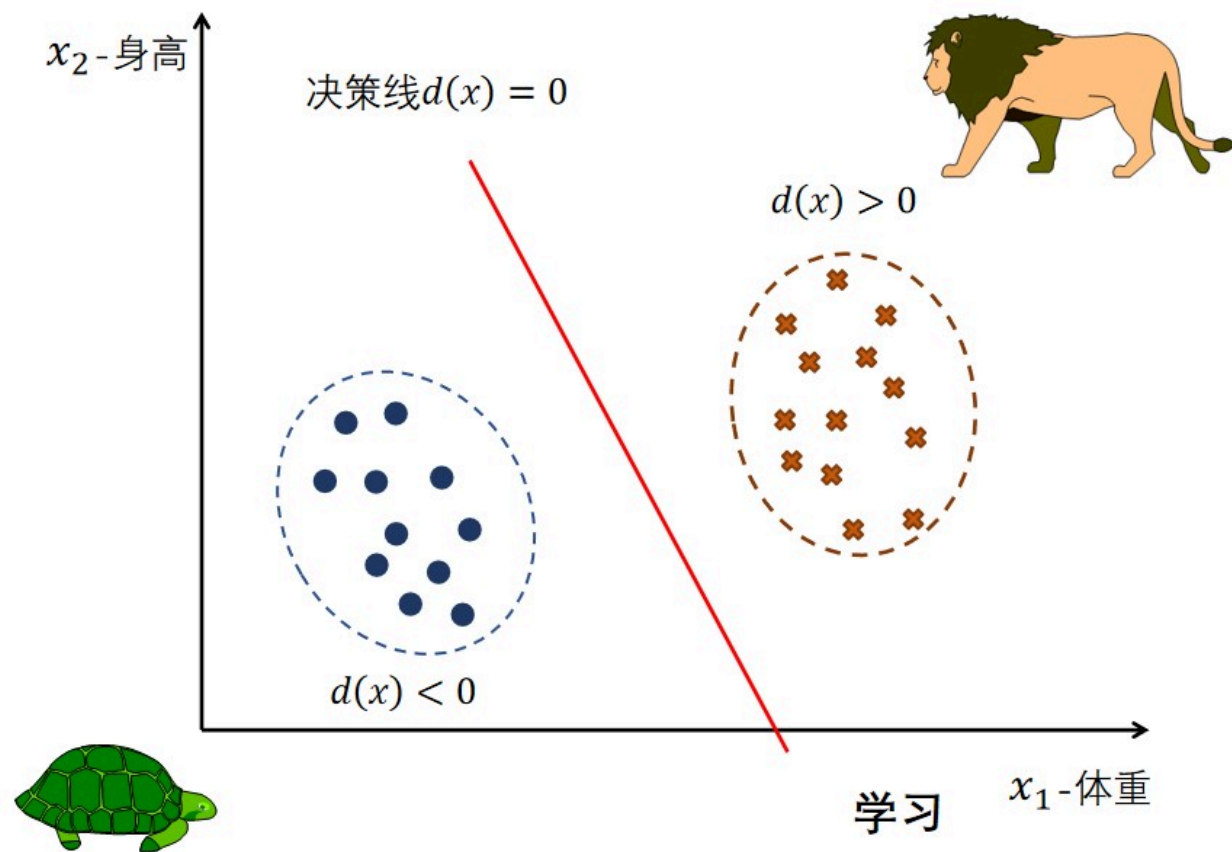
目录

- 准备知识
- 问题描述
- 感知器
- 深度神经网络
- 深度卷积神经网络

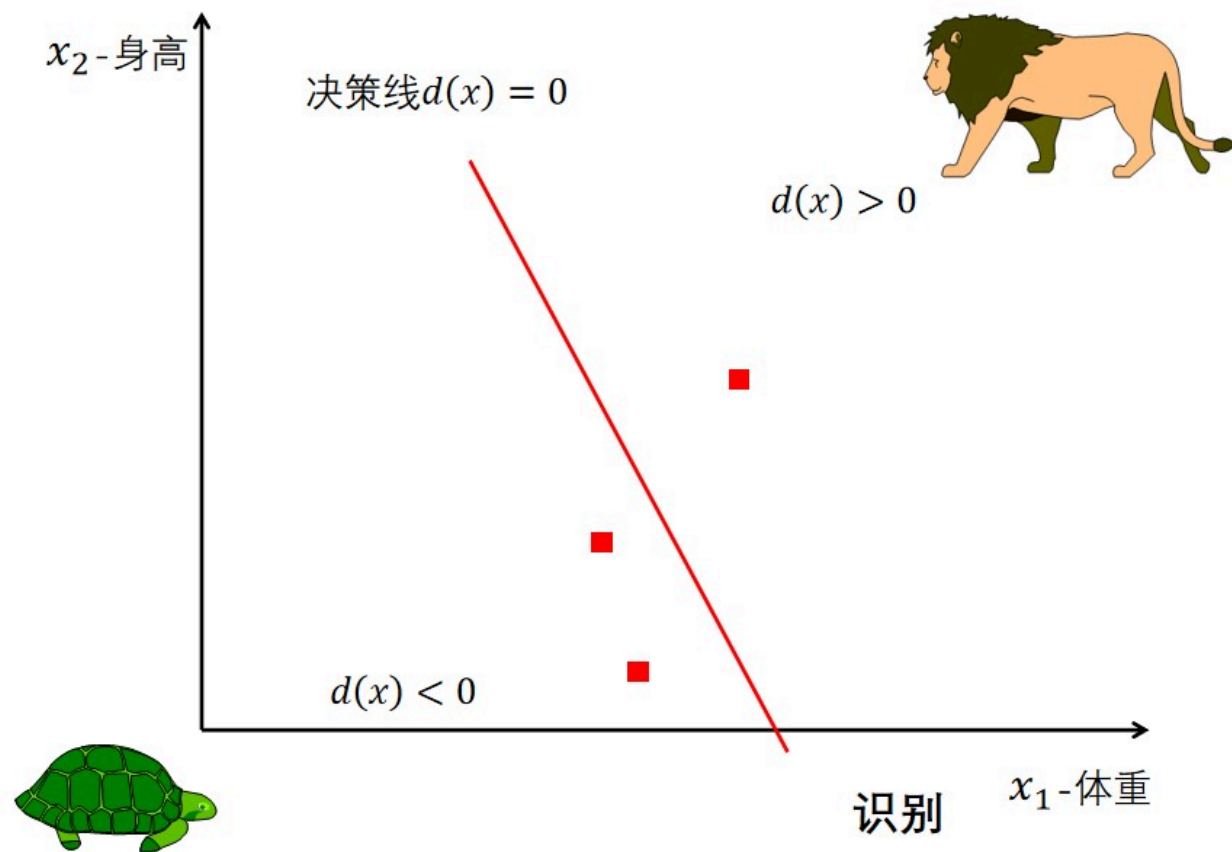


准备知识

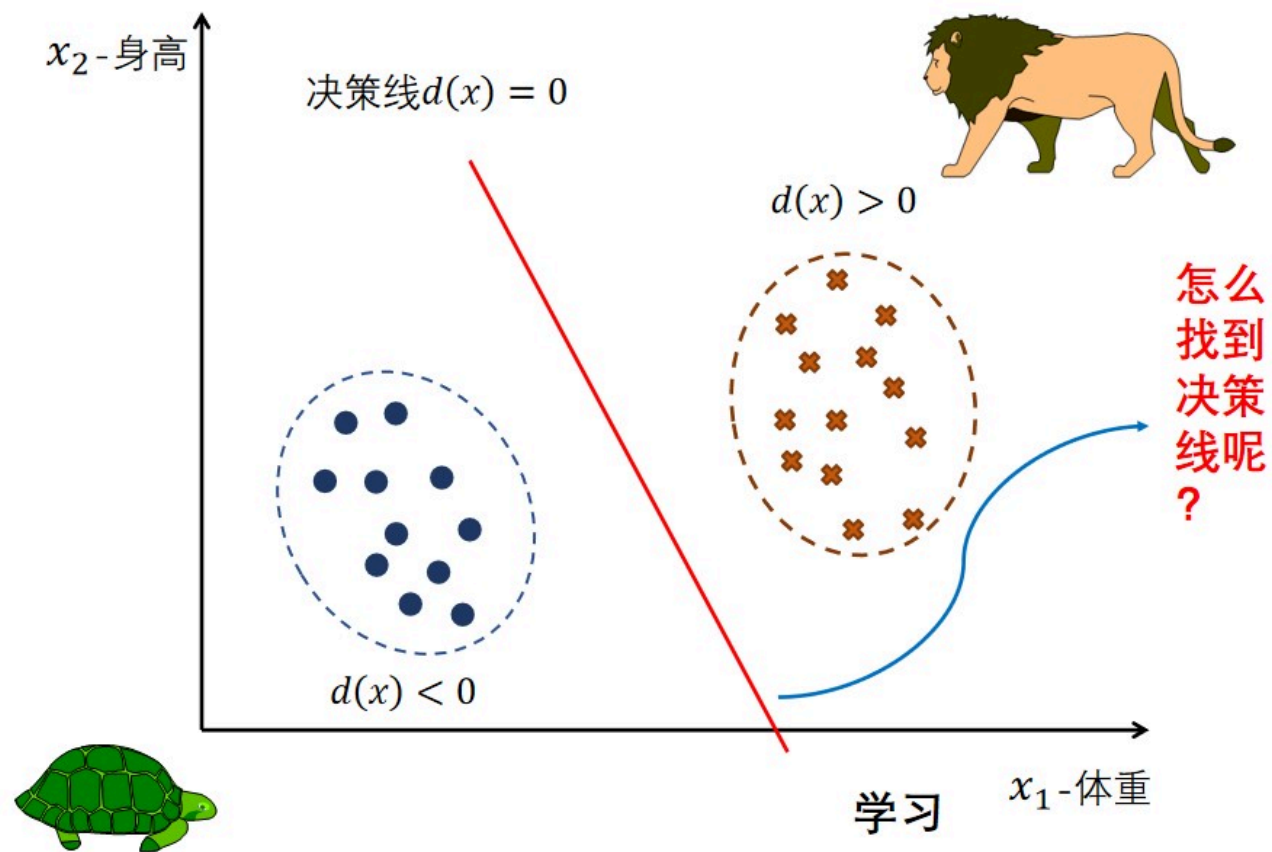
模式识别原理



模式识别原理



模式识别原理



模式识别原理

平面直线方程

➤ $x_2 = kx_1 + b, \quad k, b \in \mathbb{R}$

➤ $Ax_1 + Bx_2 + C = 0, \quad A, B, C \in \mathbb{R}$

➤ $w_0 + w_1x_1 + w_2x_2 = 0, \quad w_0, w_1, w_2 \in \mathbb{R}$

$\mathbf{w} = [w_0, w_1, w_2]^T, \mathbf{x} = [1, x_1, x_2]^T$, 上式可改写为

➤ $\mathbf{w}^T \mathbf{x} = 0, \quad \mathbf{w}, \mathbf{x} \in \mathbb{R}^3$

为什么
多了个
1呢?

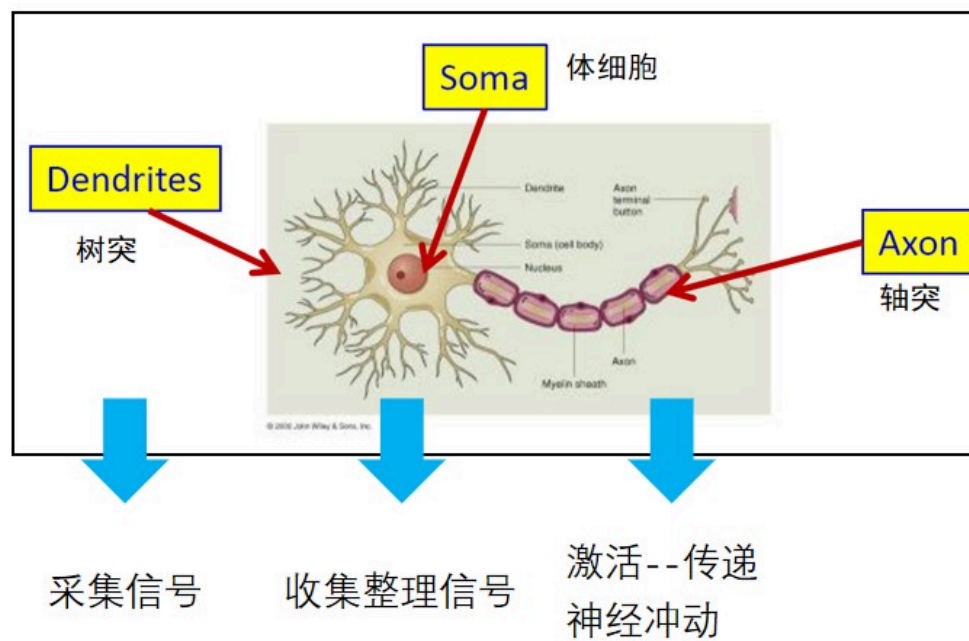
模式识别原理

问题：决策直线或者超平面如何学习得到？

- 感知器学习算法 (Perceptron Learning Algorithm, PLA)
- 支持向量机 (Support Vector Machine, SVM)
- 朴素贝叶斯分类算法 (Naive Bayes Classifier)
- ...

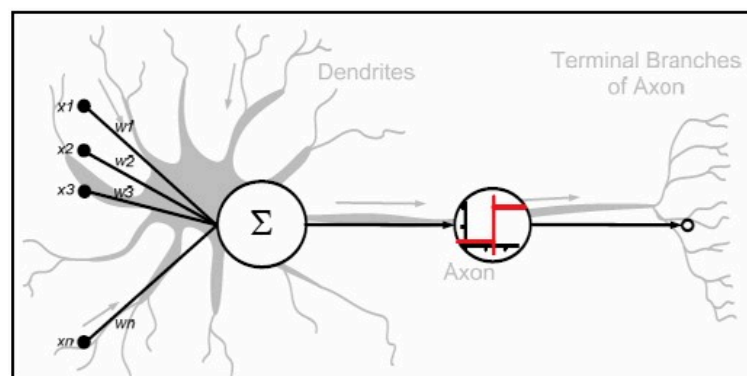
准备知识

人脑与神经元



准备知识

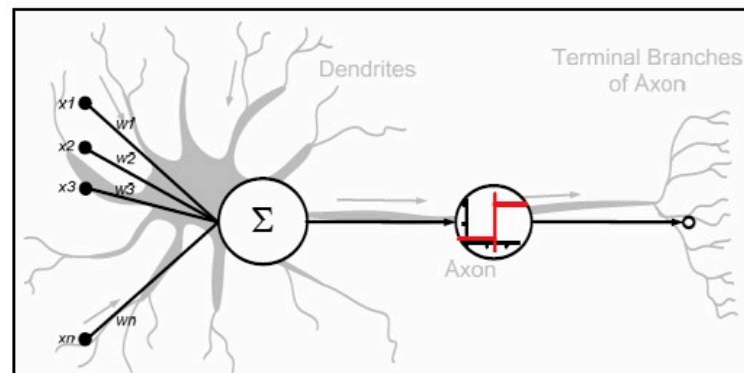
人脑与神经元



感知器提出者
Frank Rosenblatt

1958年美国心理学家
弗兰克·罗森布拉特
提出有学习能力的感
知器 (Perceptron)

准备知识



Frank Rosenblatt

Feature $\mathbf{w}^T \mathbf{x} + b$ Activation function

准备知识

1958年纽约时报关于“感知器”的报道



Perceptron
Mark I

The Perceptron was the first *artificial neural network*.

Developed by Frank Rosenblatt at the US office of Naval Research for visual recognition tasks.

The **New York Times** reported the perceptron to be: "*the embryo of an electronic computer that will be able to walk, talk, see, write, reproduce itself and be conscious of its existence.*"

准备知识

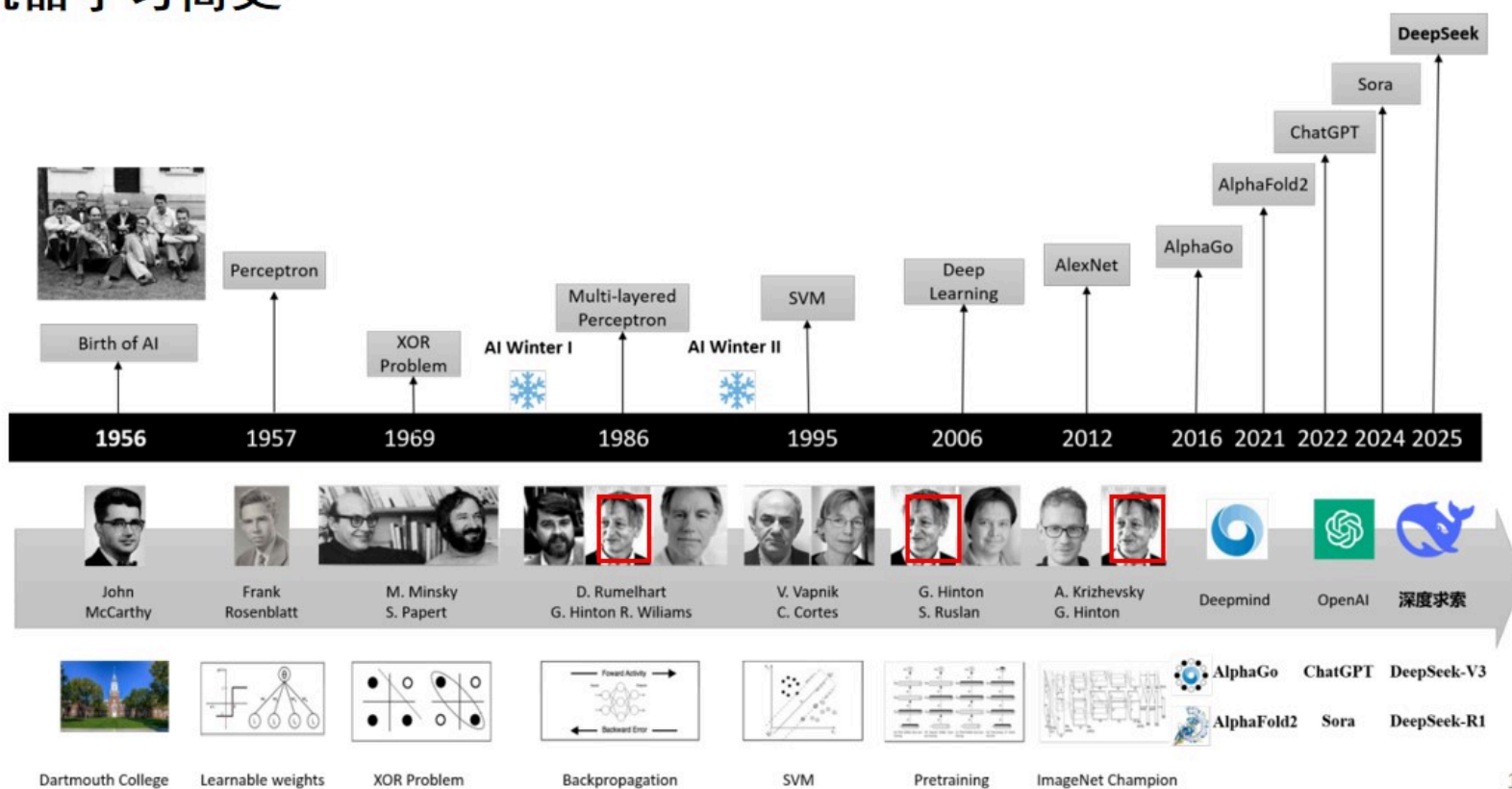
1956年人工智能的创立

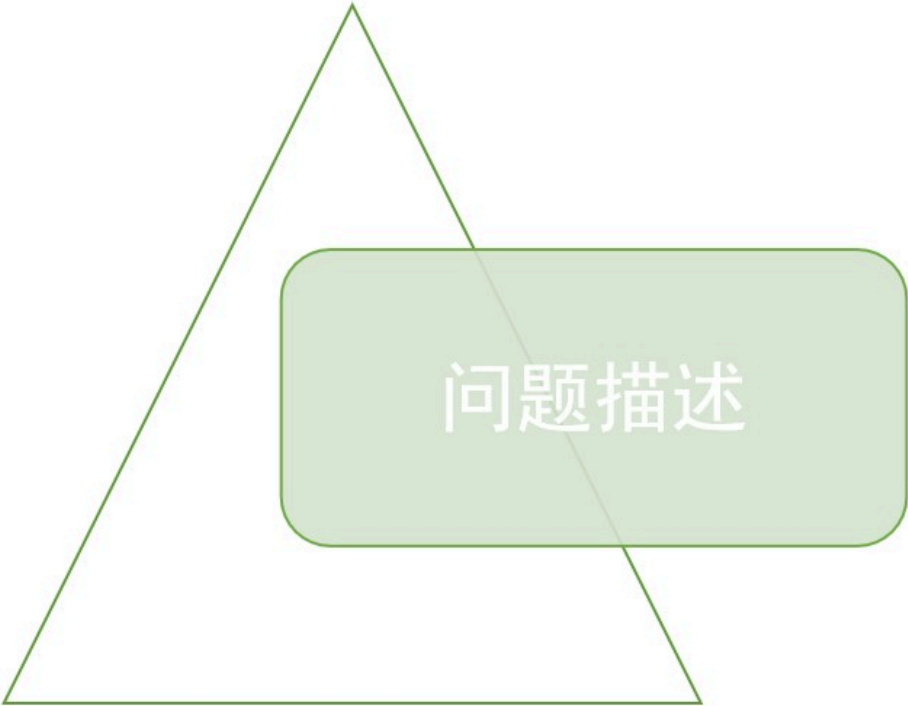


1956年，于达特茅斯学院，Allen Newell (CMU)，Herbert Simon (CMU)，John McCarthy (MIT)，Marvin Minsky(MIT) and Arthur Samuel(IBM) 创立人工智能

准备知识

机器学习简史





问题描述

问题描述

模式分类问题的目标为：设计智能算法自动将新的测试样本准确划分到真实的类别。

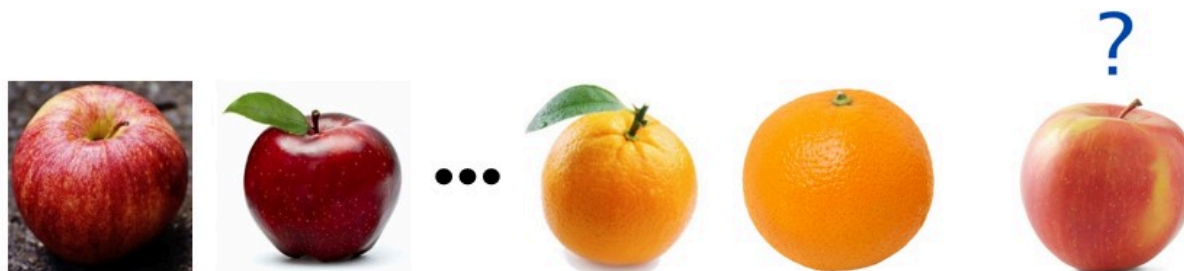


图1：模式分类的应用：水果识别，即给定多个类别水果的图像，准确识别任意新的未知类别的水果。

问题描述

模式分类问题的目标为：设计智能算法自动将新的测试样本准确划分到真实的类别。

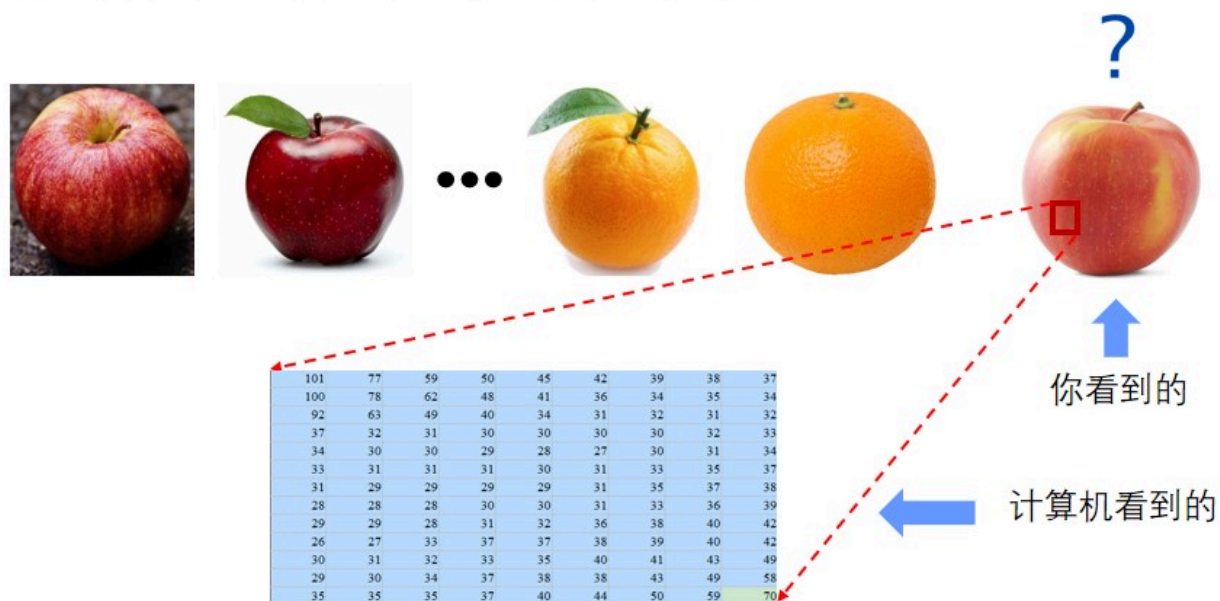


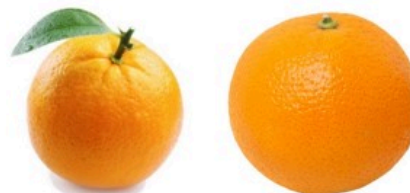
图1：模式分类的应用：水果识别，即给定多个水果的图像，准确识别任意新的未知类别的水果。

问题描述

训练样本
Training sample



标记为1



标记为-1

测试样本
Testing sample

1 还是 -1

?



问题描述



$(\mathbf{x}_1, y_1)$

特征：例如，

- 重量，长度，颜色，...
- 原始图像矩阵
- PCA特征，LBP特征，...

类别标记：1

$$\mathbf{x}_1 = \begin{bmatrix} x_{1,0} \\ x_{1,1} \\ x_{1,2} \\ \vdots \\ x_{1,d} \end{bmatrix}$$



$(\mathbf{x}_2, y_2)$

类别标记：-1

$$x_{1,0} = 1$$

问题描述



...

$(\mathbf{x}_1, y_1)$

$(\mathbf{x}_2, y_2)$

$(\mathbf{x}_3, y_3)$

$(\mathbf{x}_4, y_4)$

...

?



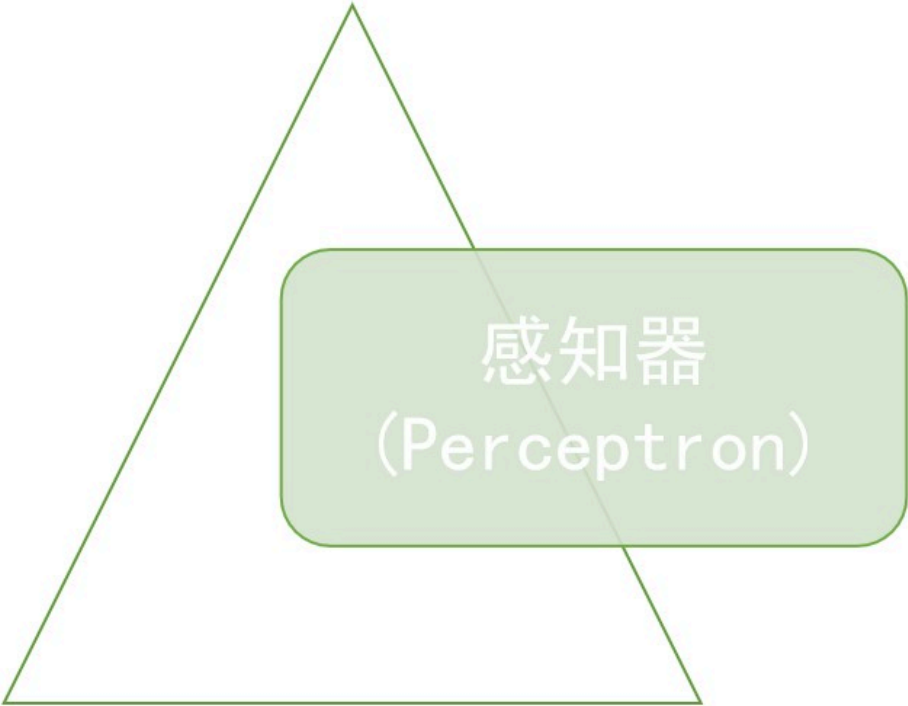
$(\mathbf{x}, ?)$

问题描述

问题公式化

符号	含义
2	类别数
n	训练样本个数
$\mathbf{x}_i$	第 i 个训练样本特征
$\mathbf{x} \in \mathbb{R}^d$	未知类别的测试样本

给定来自2类的 n 个训练样本 $\mathbf{x}_1, \dots, \mathbf{x}_n$ ，模式分类问题的目标为准确估计任意新的测试样本 $\mathbf{x}$ 的真实类别。



感知器
(Perceptron)

A diagram showing a green rounded rectangle labeled '感知器 (Perceptron)' partially overlapping a green triangle. The rectangle is positioned to the right of the triangle's base, with its left side cutting into the triangle's interior.

感知器--算法原理

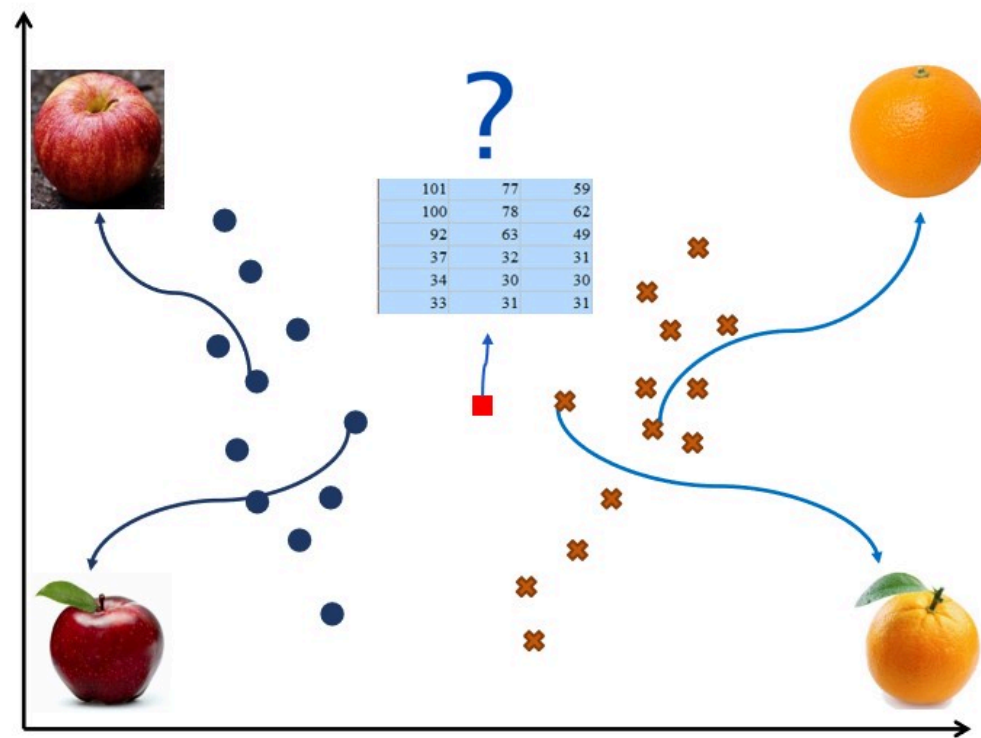
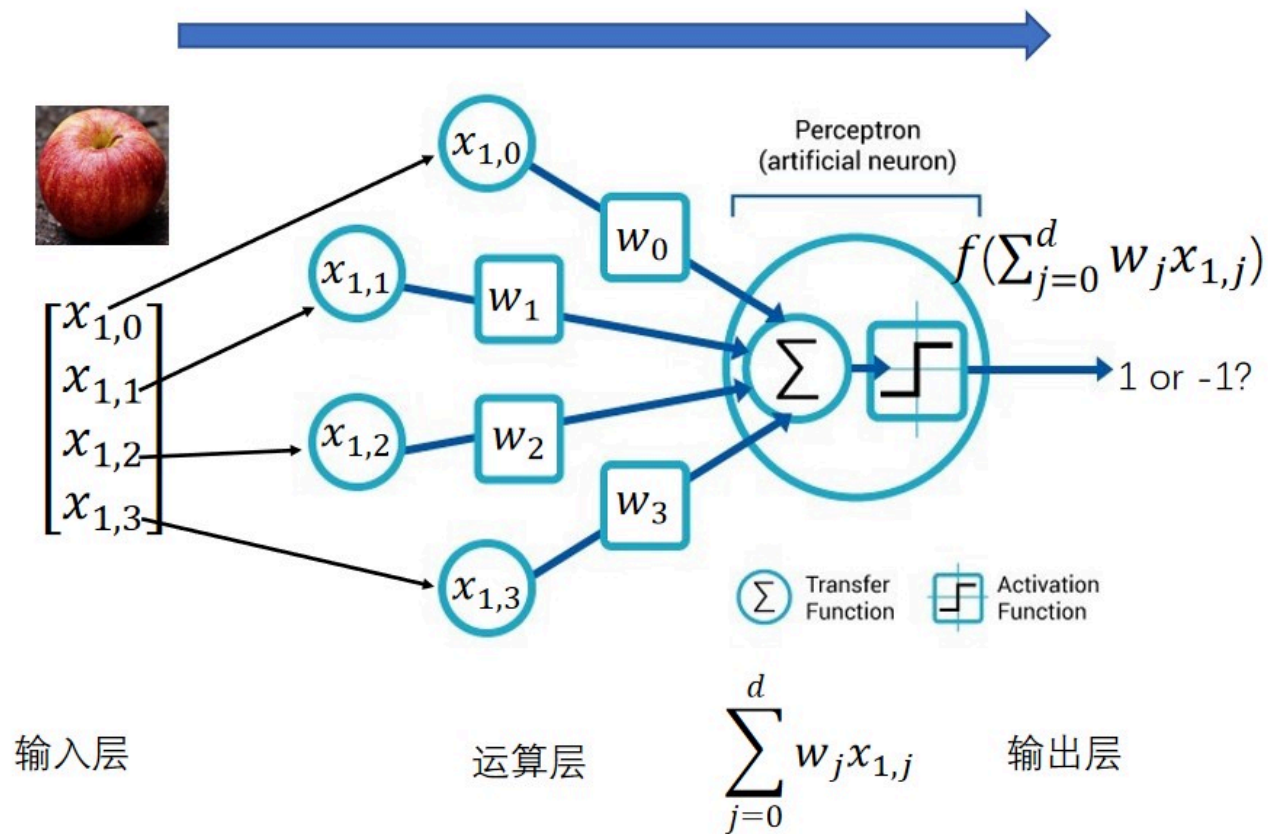


图2：模式分类问题的目标为将待识别测试样本划分到准确的类别中。

感知器--算法原理

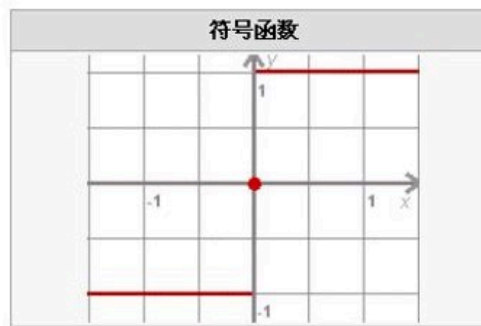


感知器--算法原理

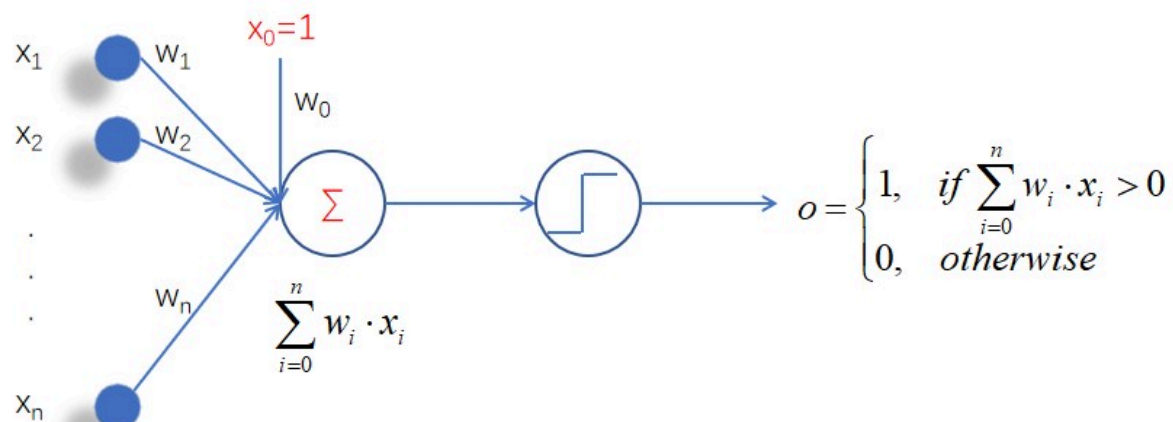
激活函数: $f: \mathbb{R} \rightarrow \mathbb{R}$



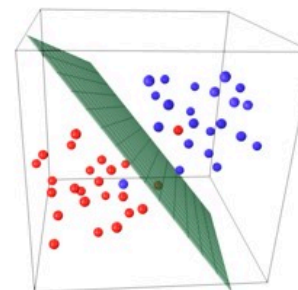
$$f(x) = \text{sign}(x) = \begin{cases} -1, & x < 0 \\ 1, & x \geq 0 \end{cases}$$



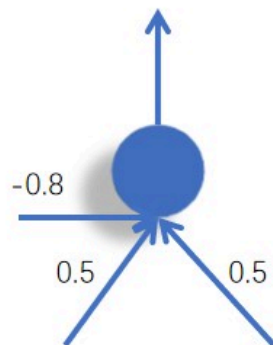
感知器--算法原理



$$o(x_1, \dots, x_n) = \begin{cases} 1, & \text{if } w_0 + w_1 \cdot x_1 + \dots + w_n \cdot x_n > 0 \\ 0, & \text{otherwise} \end{cases}$$

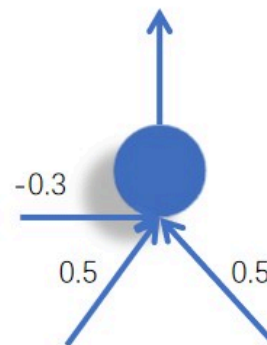


感知器--算法原理



AND

Input		Σ	Output
0	0	-0.8	0
0	1	-0.3	0
1	0	-0.3	0
1	1	0.2	1



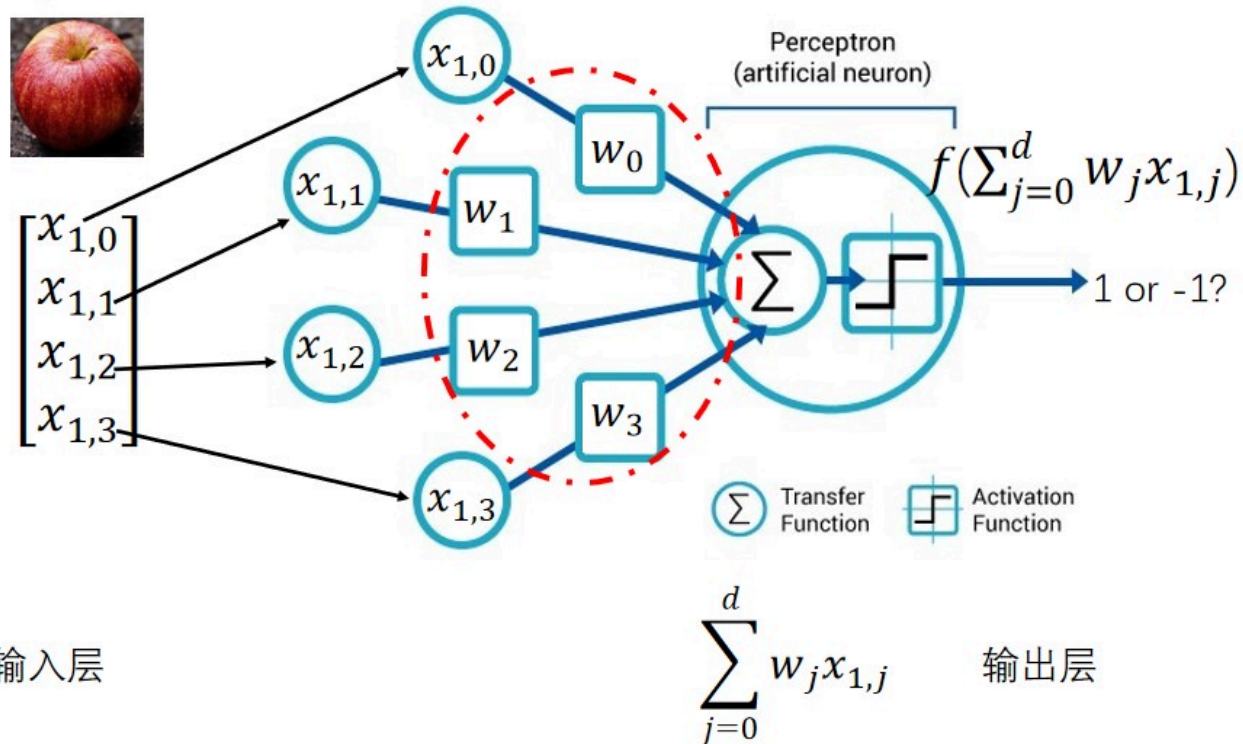
OR

Input		Σ	Output
0	0	-0.3	0
0	1	0.2	1
1	0	0.2	1
1	1	0.7	1

感知器--算法原理

要学习什么呢？感知器中哪部分是未知的呢

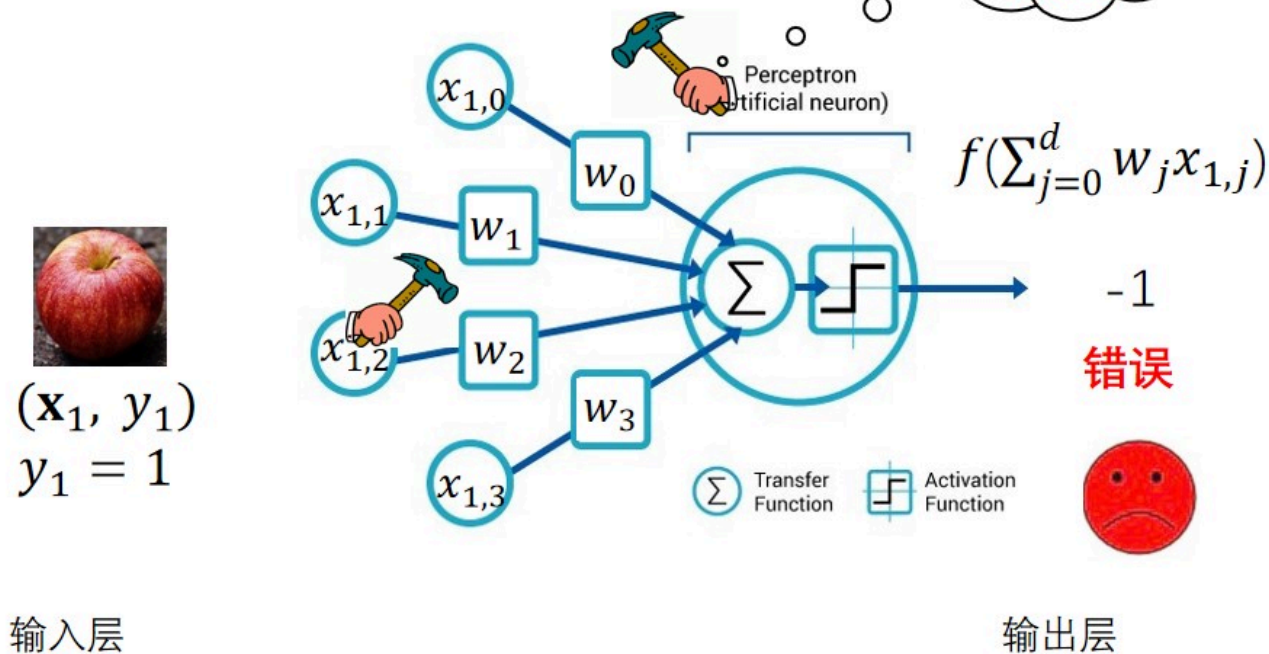
?



感知器--算法原理

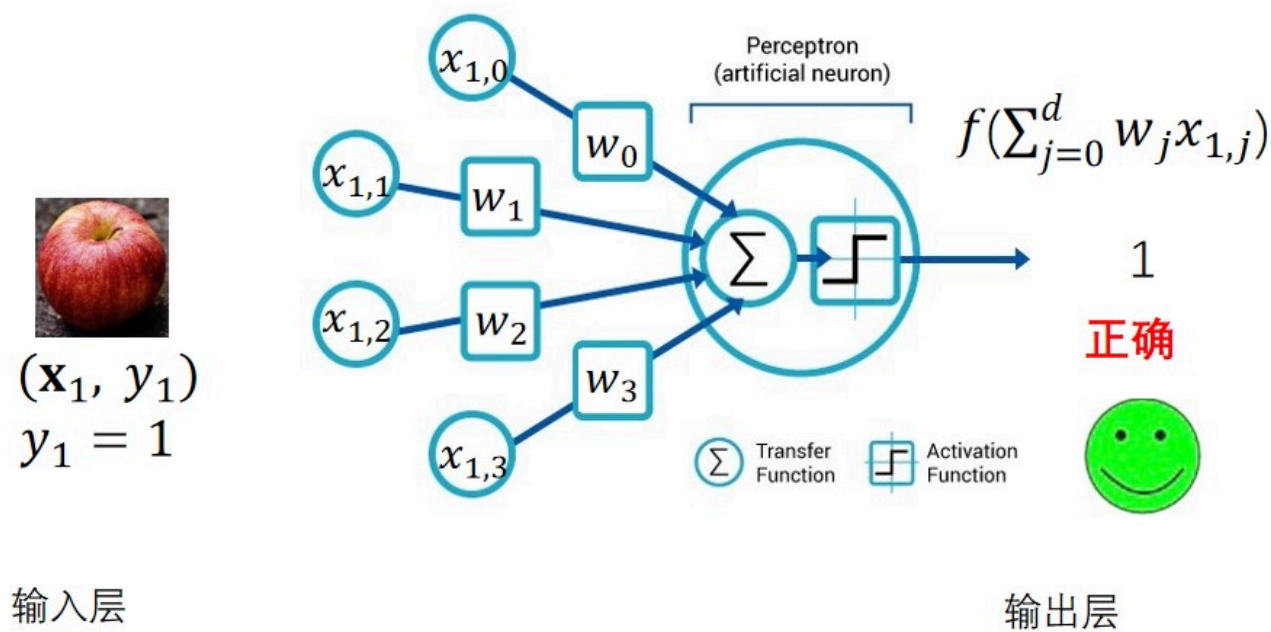
学习：权重向量 $\mathbf{w} = [w_0, w_1, \dots, w_d]$

更新
权重



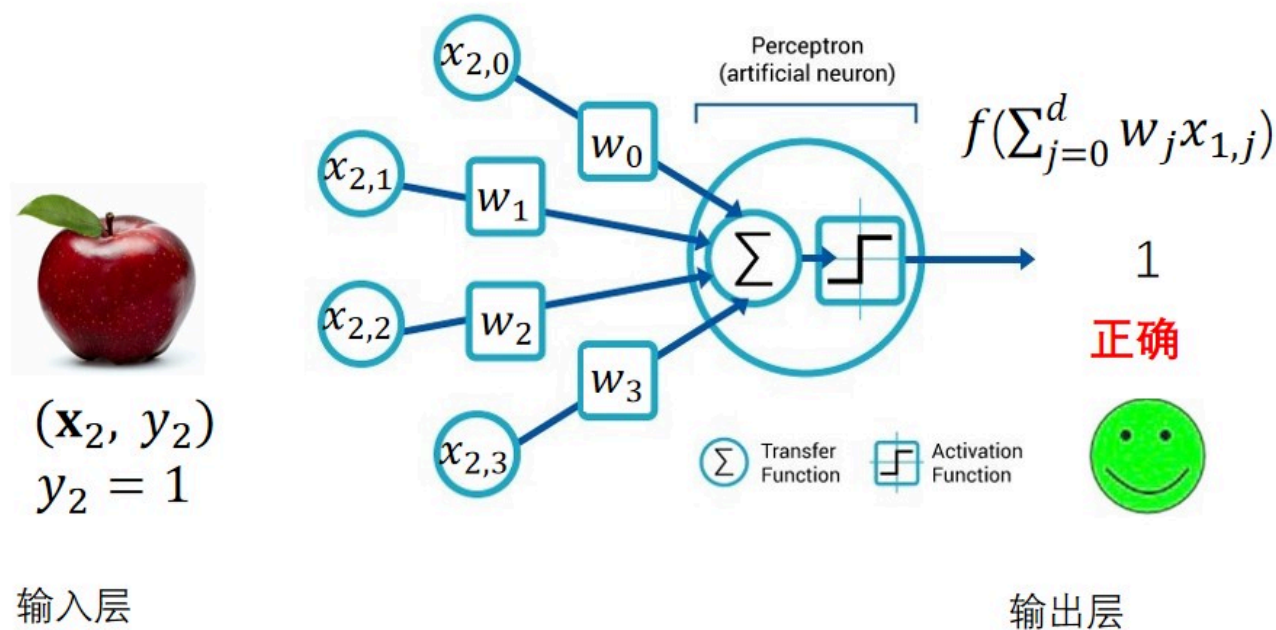
感知器--算法原理

学习：权重向量 $\mathbf{w} = [w_0, w_1, \dots, w_d]$



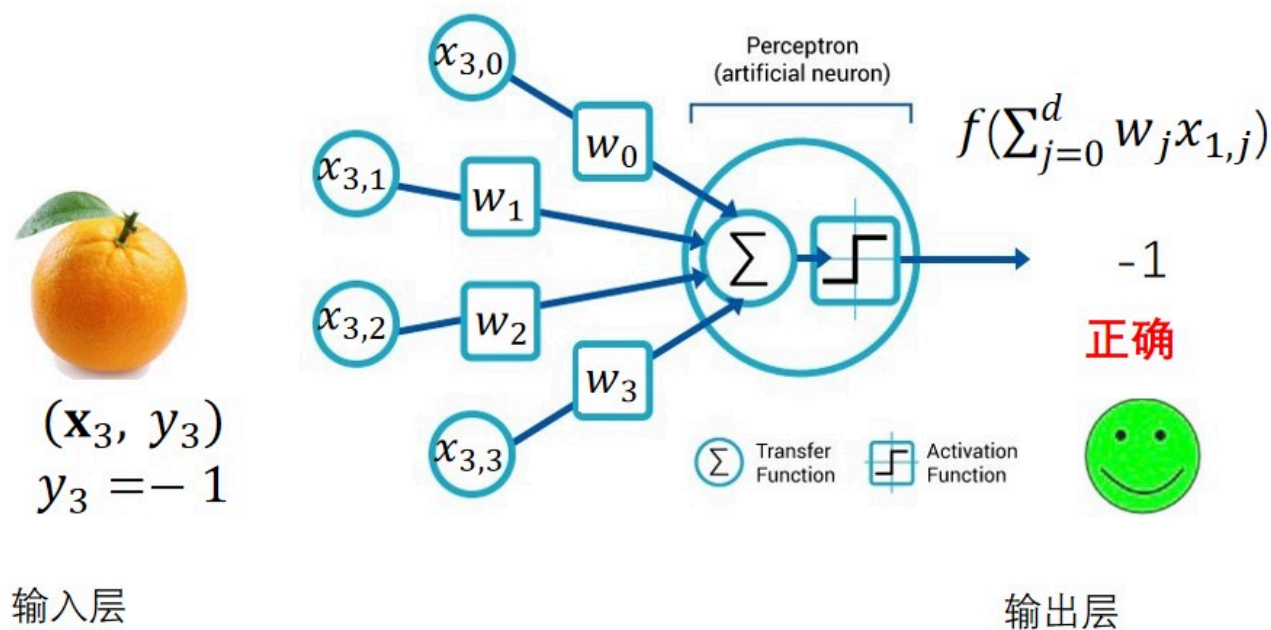
感知器--算法原理

学习：权重向量 $\mathbf{w} = [w_0, w_1, \dots, w_d]$



感知器--算法原理

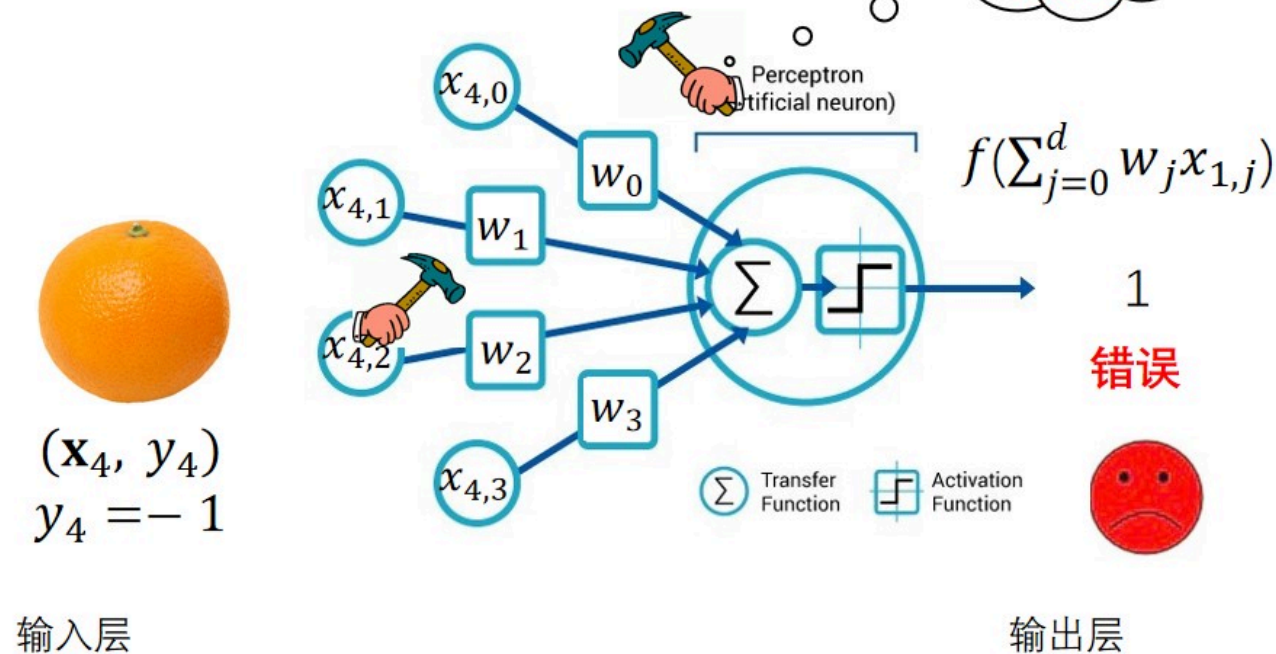
学习：权重向量 $\mathbf{w} = [w_0, w_1, \dots, w_d]$



感知器--算法原理

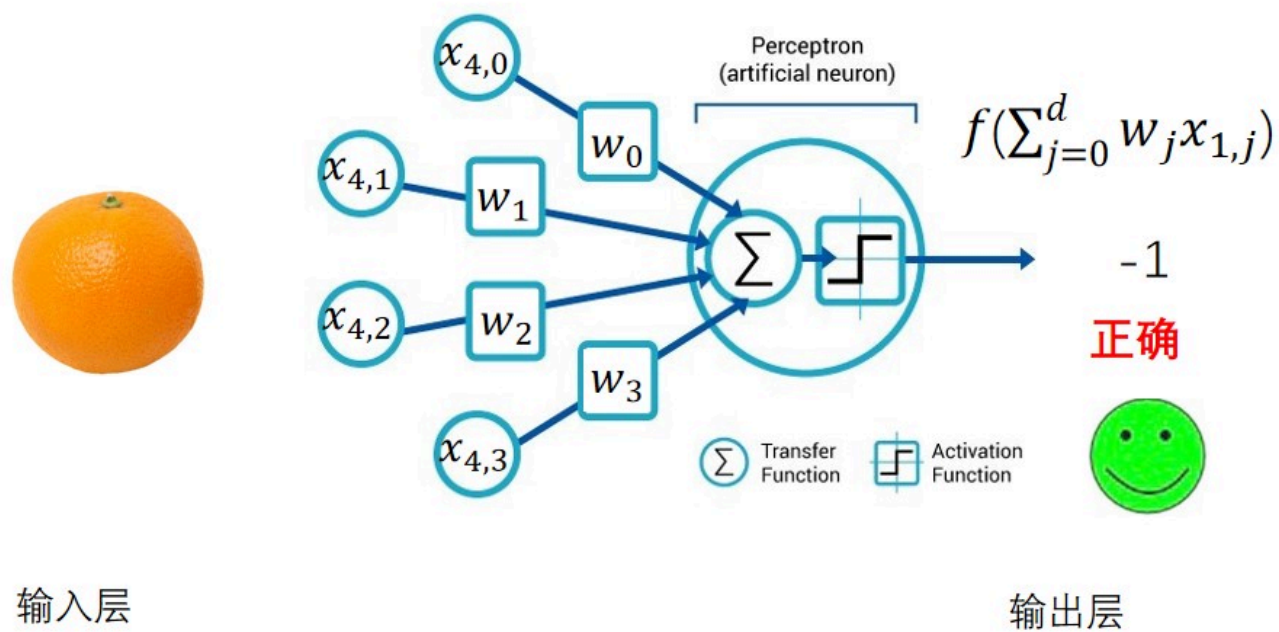
学习：权重向量 $\mathbf{w} = [w_0, w_1, \dots, w_d]$

更新
权重



感知器--算法原理

学习：权重向量 $\mathbf{w} = [w_0, w_1, \dots, w_d]$



感知器--算法原理

输入：训练样本 $(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_n, y_n)$
新的测试样本 $\mathbf{x}$

初始化：随机权重向量 $\mathbf{w}$ ，学习率 η

对于所有 n 个训练样本，执行下面步骤

步骤一：计算感知器估计结果

$$\hat{y}_i = f(\sum_{j=0}^d w_j x_{i,j}) = f(\mathbf{w}^T \mathbf{x}_i)$$
$$i = 1, 2, \dots, n$$

感知器--算法原理

输入： 训练样本 $(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_n, y_n)$
新的测试样本 $\mathbf{x}$

初始化： 随机权重向量 $\mathbf{w}$ ，学习率 η

对于所有 n 个训练样本，执行下面步骤

步骤二： 更新权重向量 $\mathbf{w}$

情况1：如果 $y_i = \hat{y}_i$ ，不更新 $\mathbf{w}$

情况2：如果 $y_i \neq \hat{y}_i$ ，更新 $\mathbf{w}$ 如下

$$\mathbf{w} = \mathbf{w} + \eta y_i \mathbf{x}_i$$

$$i = 1, 2, \dots, n$$

注：有些文献采用 $\mathbf{w} = \mathbf{w} + \eta(y_i - \hat{y}_i)\mathbf{x}_i$

感知器--算法原理

输入： 训练样本 $(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_n, y_n)$
新的测试样本 $\mathbf{x}$

初始化： 随机权重向量 $\mathbf{w}$ ，学习率 η

对于所有 n 个训练样本，执行下面步骤

步骤二： 更新权重向量 $\mathbf{w}$

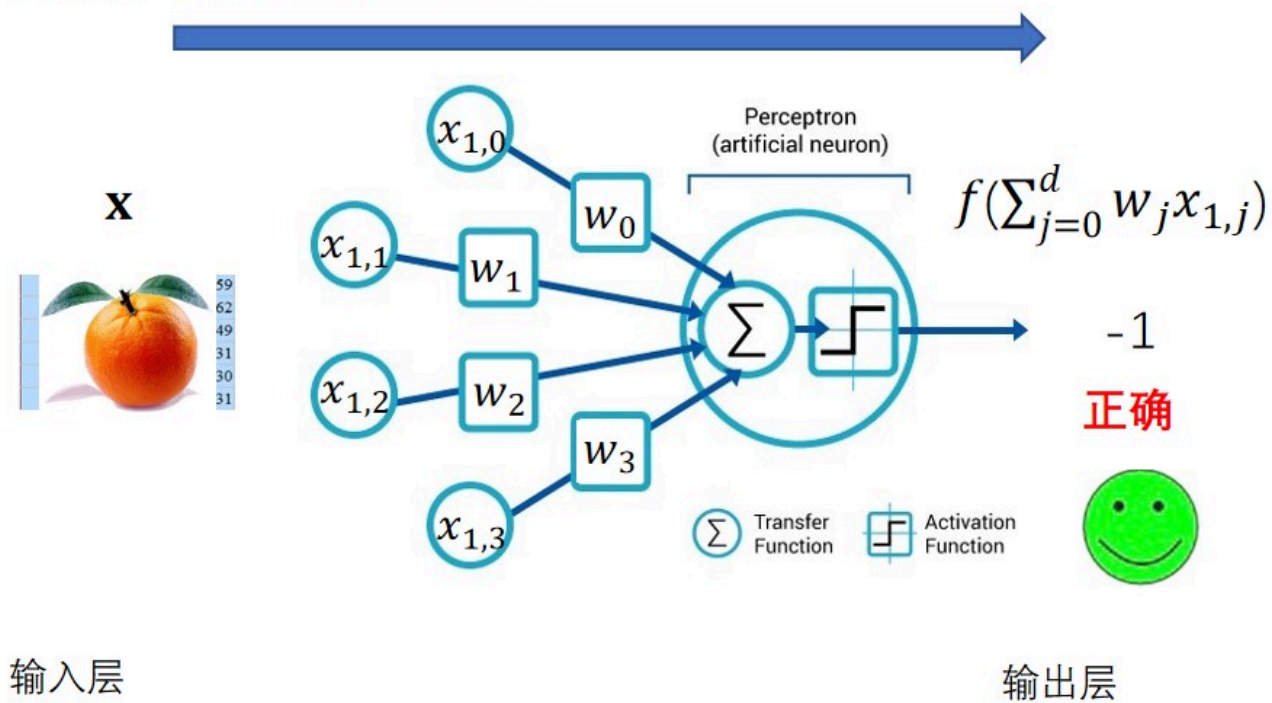
$$\mathbf{w} = \mathbf{w} + \eta y_i \mathbf{x}_i \quad \{i: y_i \neq \hat{y}_i\}$$

重复上述步骤直到对于所有训练样本没有分类错误
或者达到一定的迭代次数

输出： 权重向量 $\mathbf{w}$

感知器--算法原理

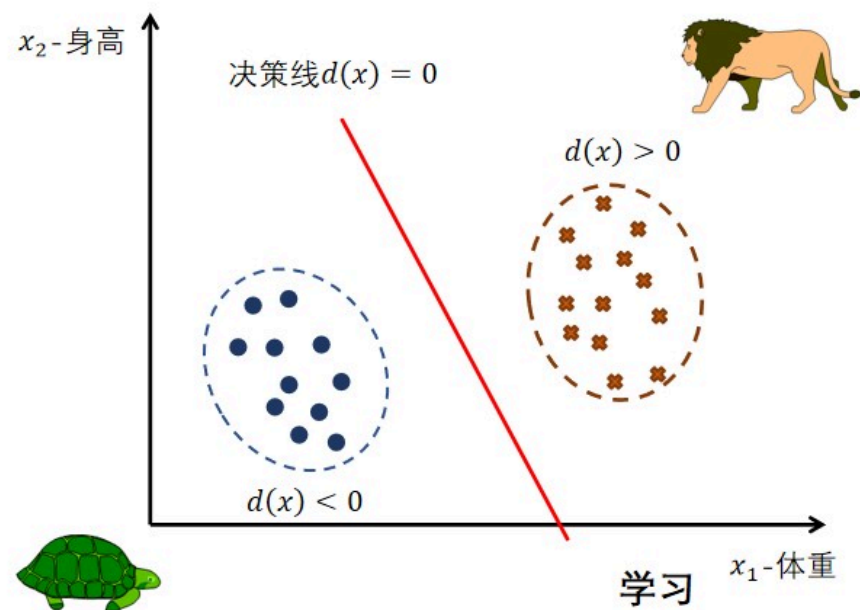
识别：新的测试样本x



思考

◆ 为什么更新权重向量 $\mathbf{w}$ 的公式为：

$$\mathbf{w} = \mathbf{w} + \eta y_i \mathbf{x}_i \quad \{i: y_i \neq \hat{y}_i\}?$$



39

思考

◆ 知其然，知其所以然

人跟机器最大的不同在于，机器只能做到前者，而人可以却可以做到后者。

40

思考

➤ 情况1: $y_i = 1, \hat{y}_i = -1$

更新前:

$\hat{y}_i = \text{sign}(\mathbf{w}^T \mathbf{x}_i) = -1$, 这说明 $\mathbf{w}^T \mathbf{x}_i < 0$



更新后:

$$\begin{aligned}\hat{y}_i^{new} &= \text{sign}((\mathbf{w} + \eta y_i \mathbf{x}_i)^T \mathbf{x}_i) \\ &= \text{sign}(\mathbf{w}^T \mathbf{x}_i + \eta y_i \mathbf{x}_i^T \mathbf{x}_i) \\ &= \text{sign}(\mathbf{w}^T \mathbf{x}_i + \eta \|\mathbf{x}_i\|_2^2)\end{aligned}$$

可选择合适的学习率 η 使得

$$\mathbf{w}^T \mathbf{x}_i + \eta \|\mathbf{x}_i\|_2^2 > 0$$

则 $\hat{y}_i^{new} = 1 = y_i$, 分类正确。

41

思考

➤ 情况2: $y_i = -1, \hat{y}_i = 1$

更新前:

$\hat{y}_i = \text{sign}(\mathbf{w}^T \mathbf{x}_i) = 1$, 这说明 $\mathbf{w}^T \mathbf{x}_i > 0$



更新后:

$$\begin{aligned}\hat{y}_i^{new} &= \text{sign}((\mathbf{w} + \eta y_i \mathbf{x}_i)^T \mathbf{x}_i) \\ &= \text{sign}(\mathbf{w}^T \mathbf{x}_i + \eta y_i \mathbf{x}_i^T \mathbf{x}_i) \\ &= \text{sign}(\mathbf{w}^T \mathbf{x}_i - \eta \|\mathbf{x}_i\|_2^2)\end{aligned}$$

可选择合适的学习率 η 使得

$$\mathbf{w}^T \mathbf{x}_i - \eta \|\mathbf{x}_i\|_2^2 < 0$$

则 $\hat{y}_i^{new} = -1 = y_i$, 分类正确。

42

例子

Input			Target	Initial Weights			Output					Error	Correction	Final Weights		
							Individual			Sum	Final Output					
x ₀	x ₁	x ₂	t	w ₀	w ₁	w ₂	C ₀	C ₁	C ₂	S	o	E	R	w ₀	w ₁	w ₂
							x ₀ · w ₀	x ₁ · w ₁	x ₂ · w ₂	C ₀ +C ₁ +C ₂		t-o	LR x E			
1	0	0	1	0	0	0	0	0	0	0	0	1	+0.1	0.1	0	0
1	0	1	1	0.1	0	0	0.1	0	0	0.1	0	1	+0.1	0.2	0	0.1
1	1	0	1	0.2	0	0.1	0.2	0	0	0.2	0	1	+0.1	0.3	0.1	0.1
1	1	1	0	0.3	0.1	0.1	0.3	0.1	0.1	0.5	0	0	0	0.3	0.1	0.1
1	0	0	1	0.3	0.1	0.1	0.3	0	0	0.3	0	1	+0.1	0.4	0.1	0.1
1	0	1	1	0.4	0.1	0.1	0.4	0	0.1	0.5	0	1	+0.1	0.5	0.1	0.2
1	1	0	1	0.5	0.1	0.2	0.5	0.1	0	0.6	1	0	0	0.5	0.1	0.2
1	1	1	0	0.5	0.1	0.2	0.5	0.1	0.2	0.8	1	-1	-0.1	0.4	0	0.1
1	0	0	1	0.4	0	0.1	0.4	0	0	0.4	0	1	+0.1	0.5	0	0.1
.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
1	1	0	1	0.8	-2	-1	0.8	-2	0	0.6	1	0	0	0.8	-2	-1

threshold=0.5 learning rate=0.1

例子

例2：人脸识别



[1] A. Martinez and R. Benavente, "The AR face database," Computer Vision Center, Tech. Rep. 24, Jun. 1998.

例子

训练样本
Training sample



测试样本
Testing sample

标记为1 1 还是 -1

?

标记为-1



[1] A. Martinez and R. Benavente, "The AR face database," Computer Vision Center, Tech. Rep. 24, Jun. 1998.

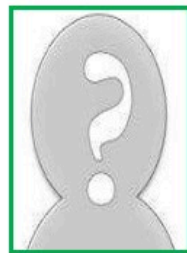
例子

例2：人脸识别

测试样本 Testing sample



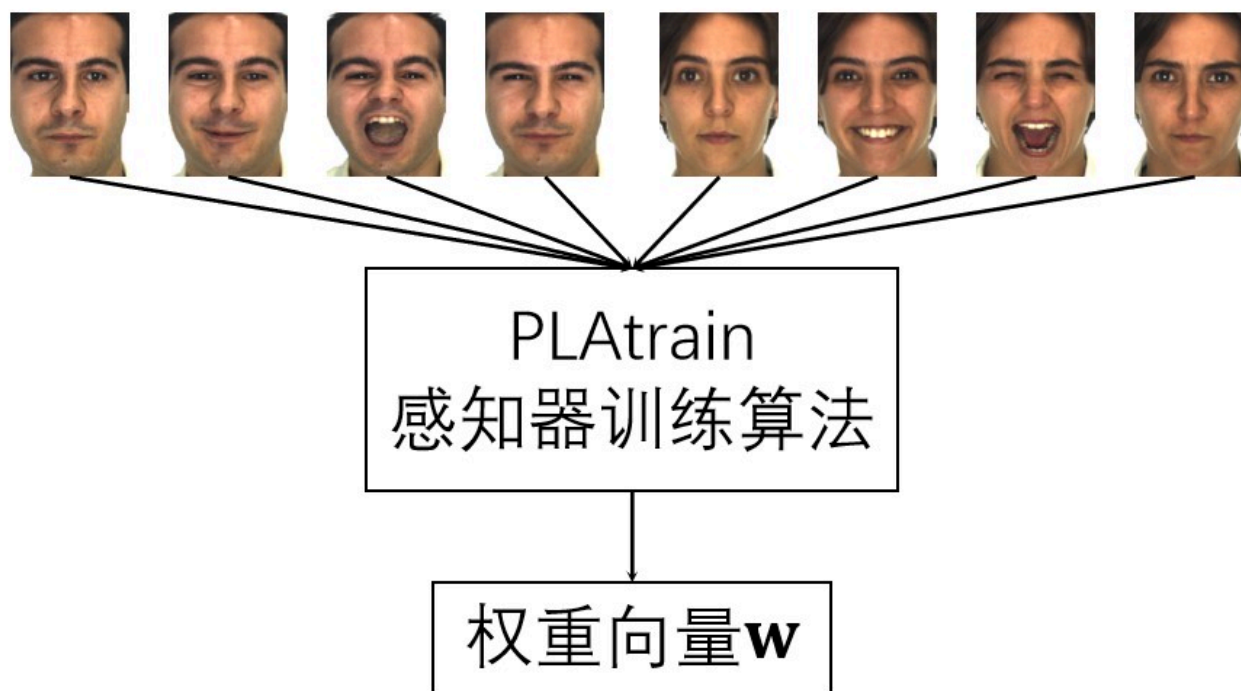
测试人脸1



测试人脸2

例子

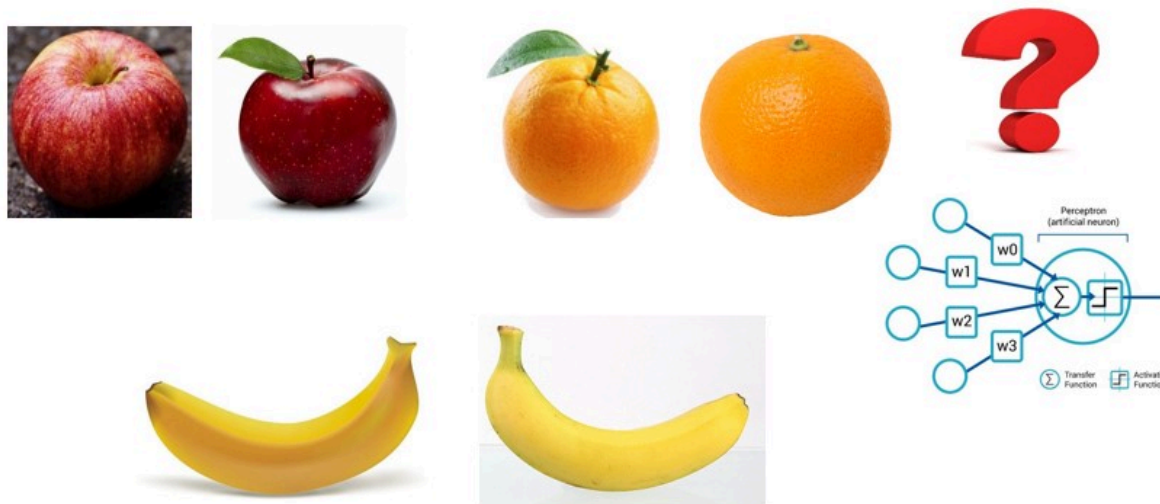
例2：人脸识别：第一步，训练，学习权重



多类别感知器

多类别感知器 (Multiclass Perceptron)

感知器本质上是一个二分类算法，如何将其推广至多类别的模式分类问题呢？



多类别感知器

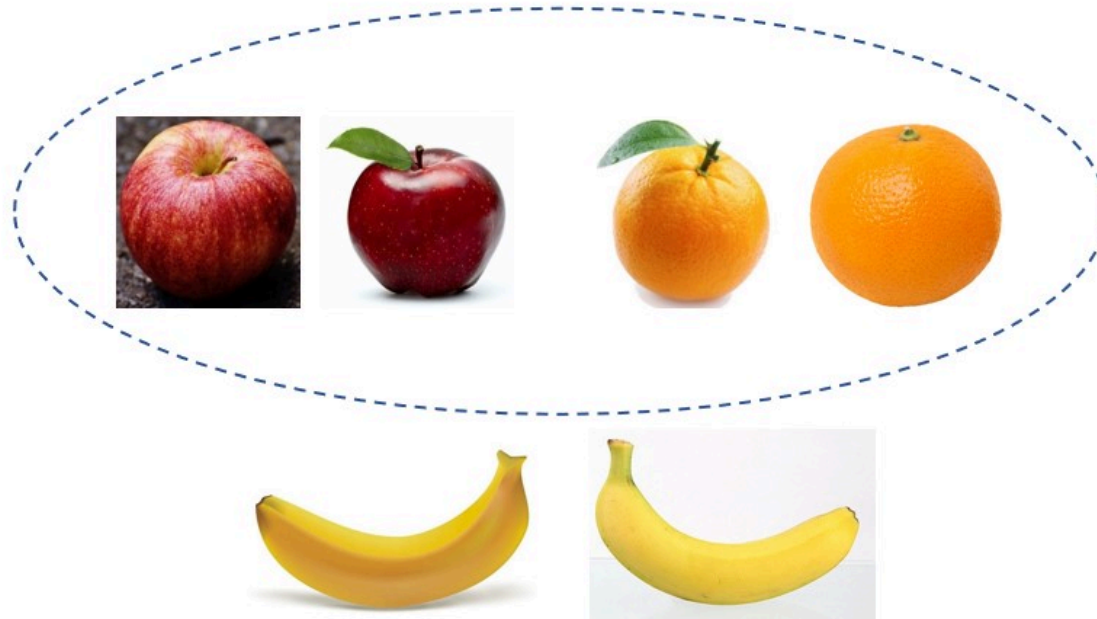
多类别感知器 (Multiclass Perceptron)

感知器本质上是一个二分类算法，如何将其推广至多类别的模式分类问题呢？

- ◆ 一对一 (One vs. One, OvO)
- ◆ 一对多 (One vs. Rest, OvR)

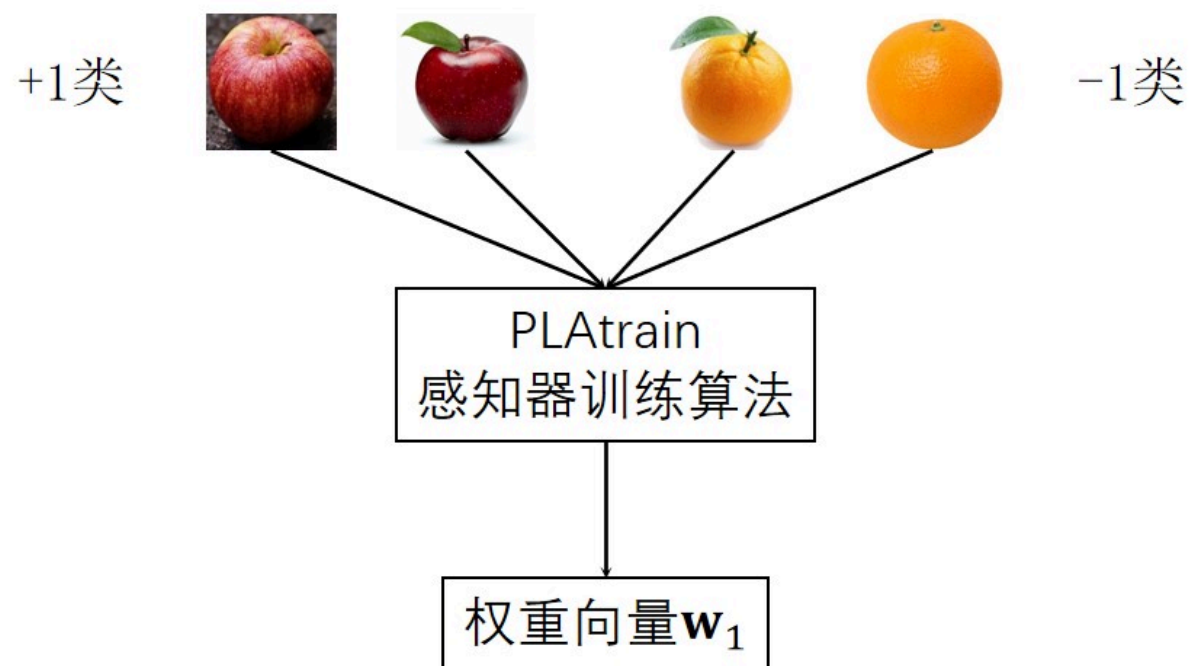
多类别感知器

◆ 一对一 (One vs. One, OvO)



多类别感知器

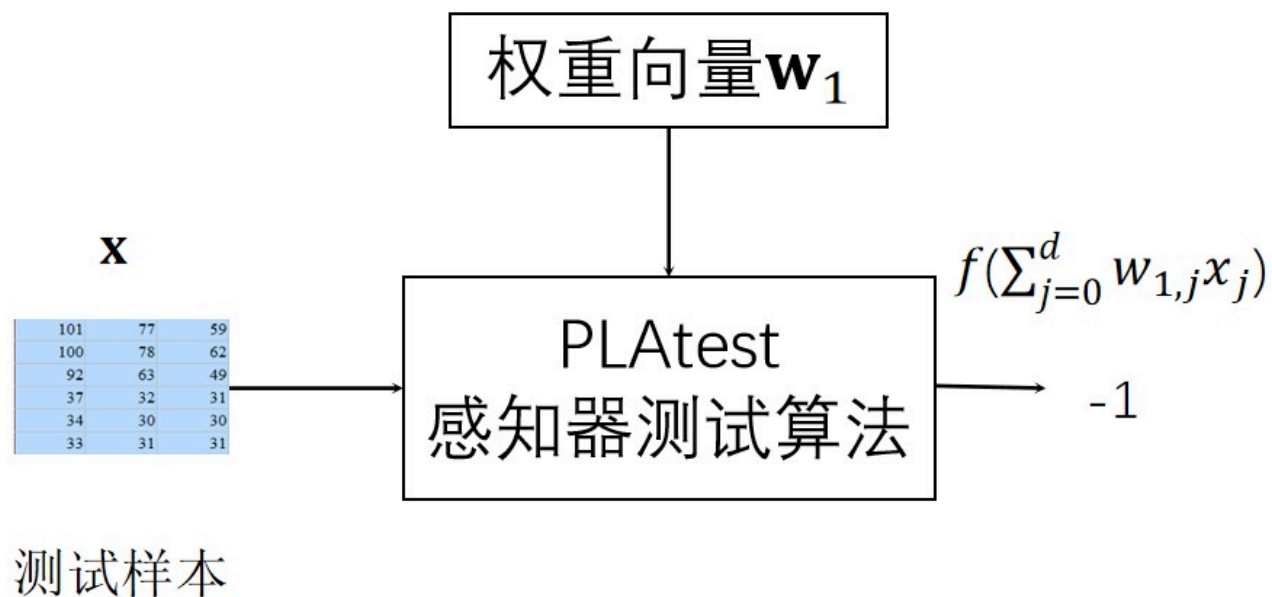
◆ 一对一 (One vs. One, OvO)



51

多类别感知器

◆ 一对一 (One vs. One, OvO)



多类别感知器

◆ 一对一 (One vs. One, OvO)

类别1



类别2

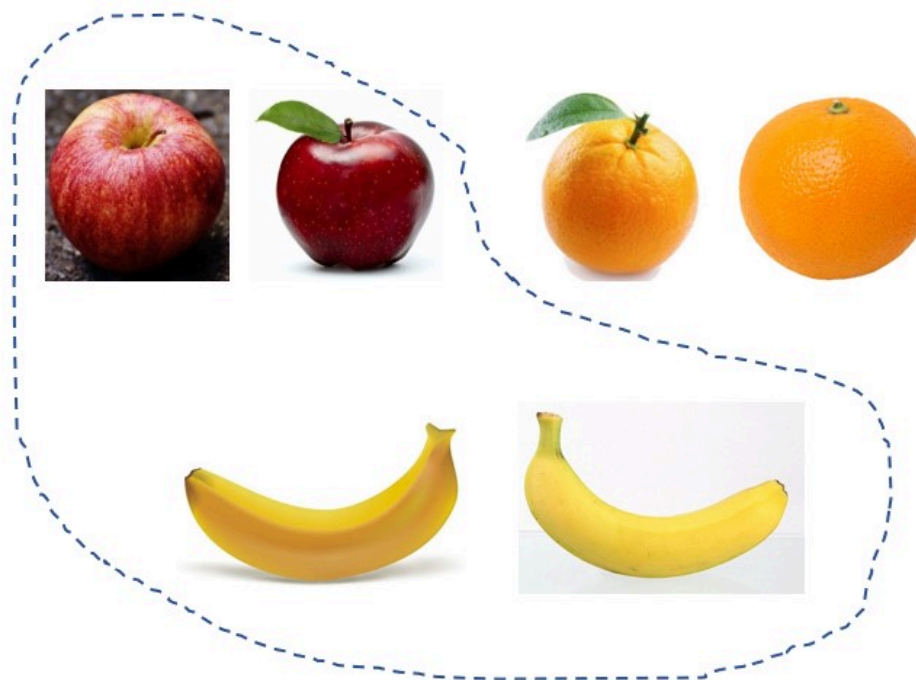


类别3



多类别感知器

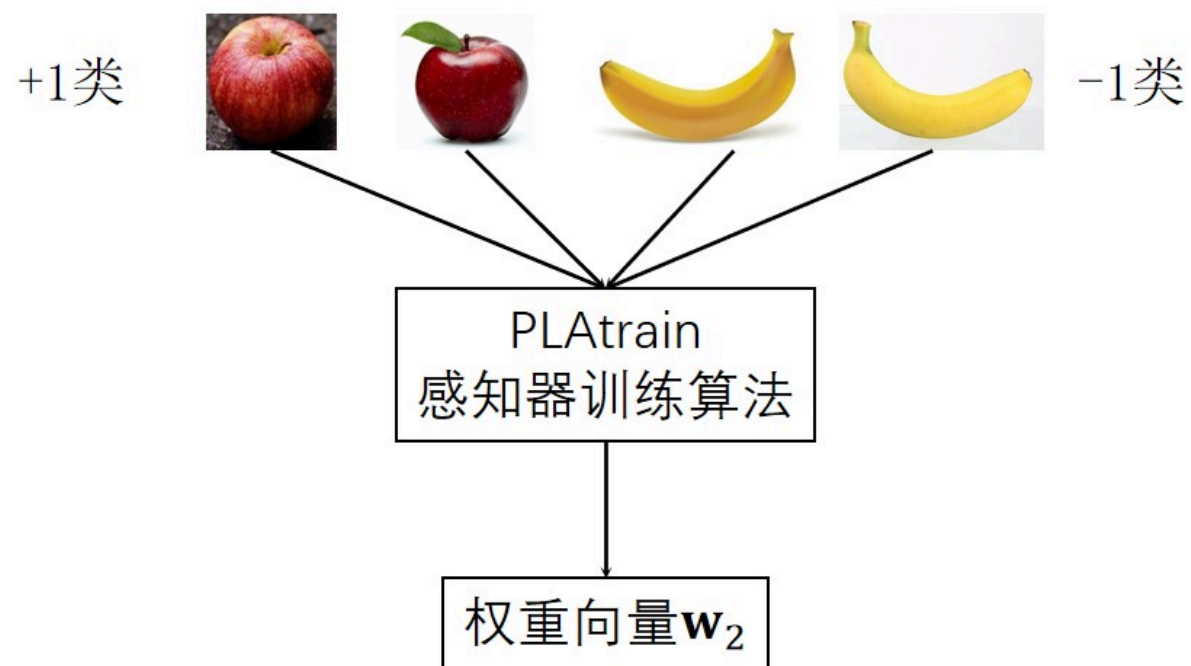
◆ 一对一 (One vs. One, OvO)



54

多类别感知器

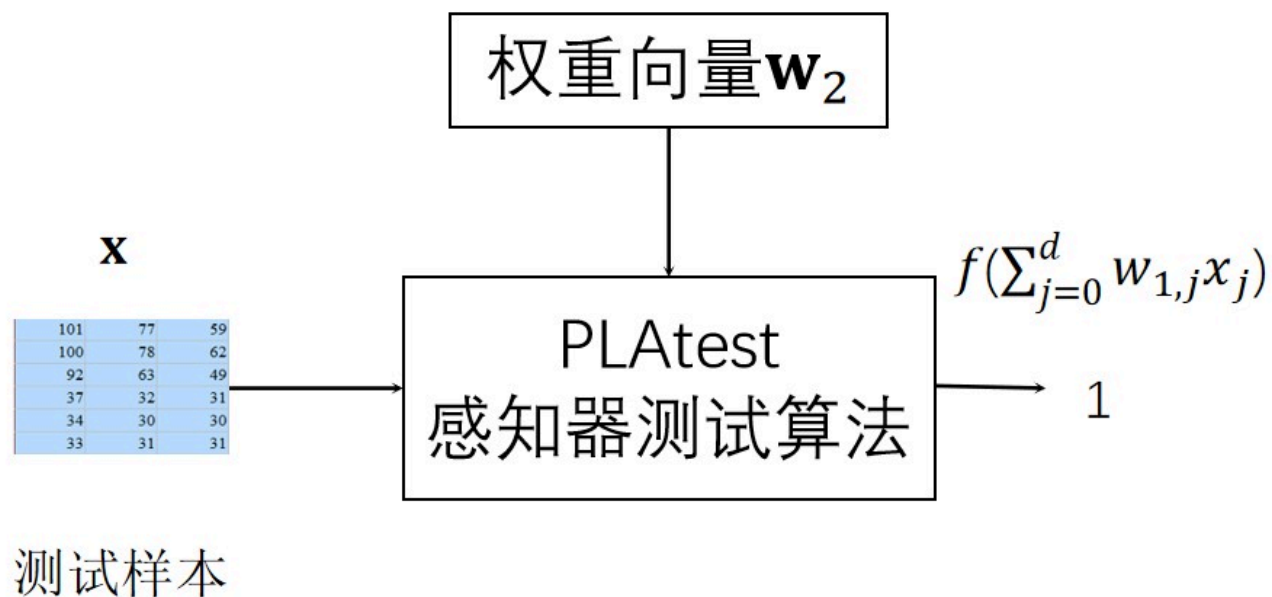
◆ 一对一 (One vs. One, OvO)



55

多类别感知器

◆ 一对一 (One vs. One, OvO)



多类别感知器

◆ 一对一 (One vs. One, OvO)

类别1



类别2

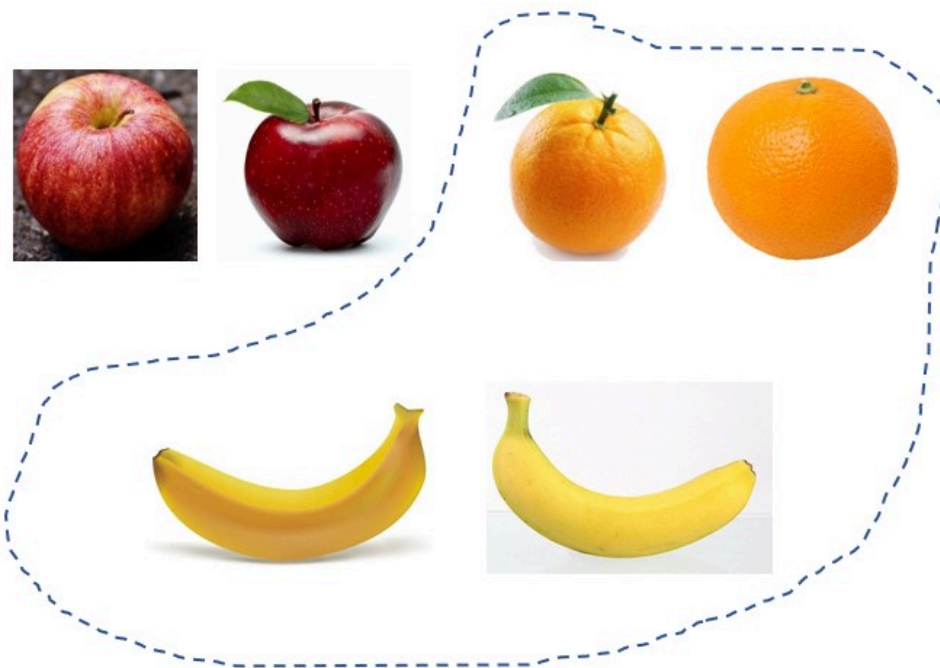


类别3



多类别感知器

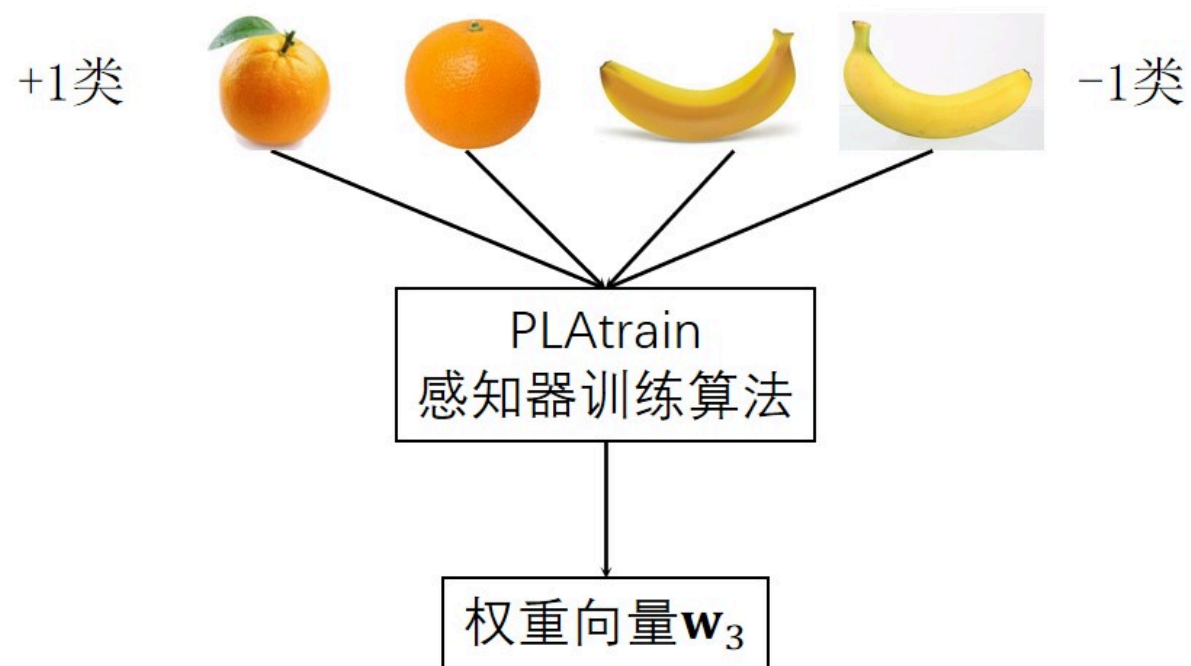
◆ 一对一 (One vs. One, OvO)



58

多类别感知器

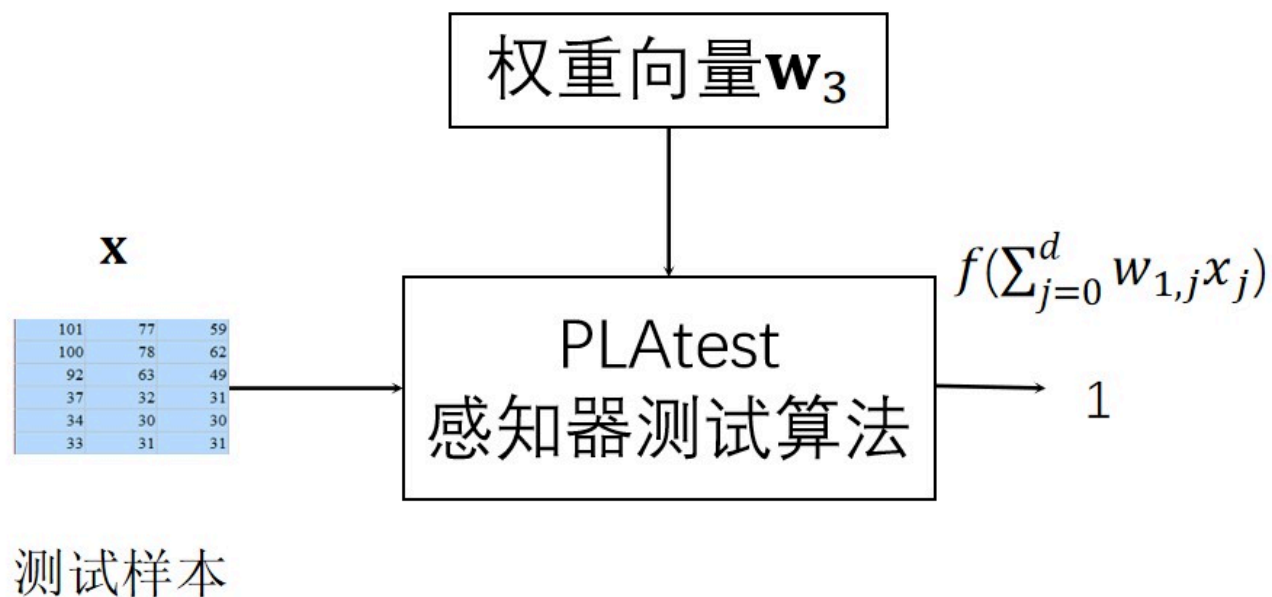
◆ 一对一 (One vs. One, OvO)



59

多类别感知器

◆ 一对一 (One vs. One, OvO)



多类别感知器

◆ 一对一 (One vs. One, OvO)



多类别感知器

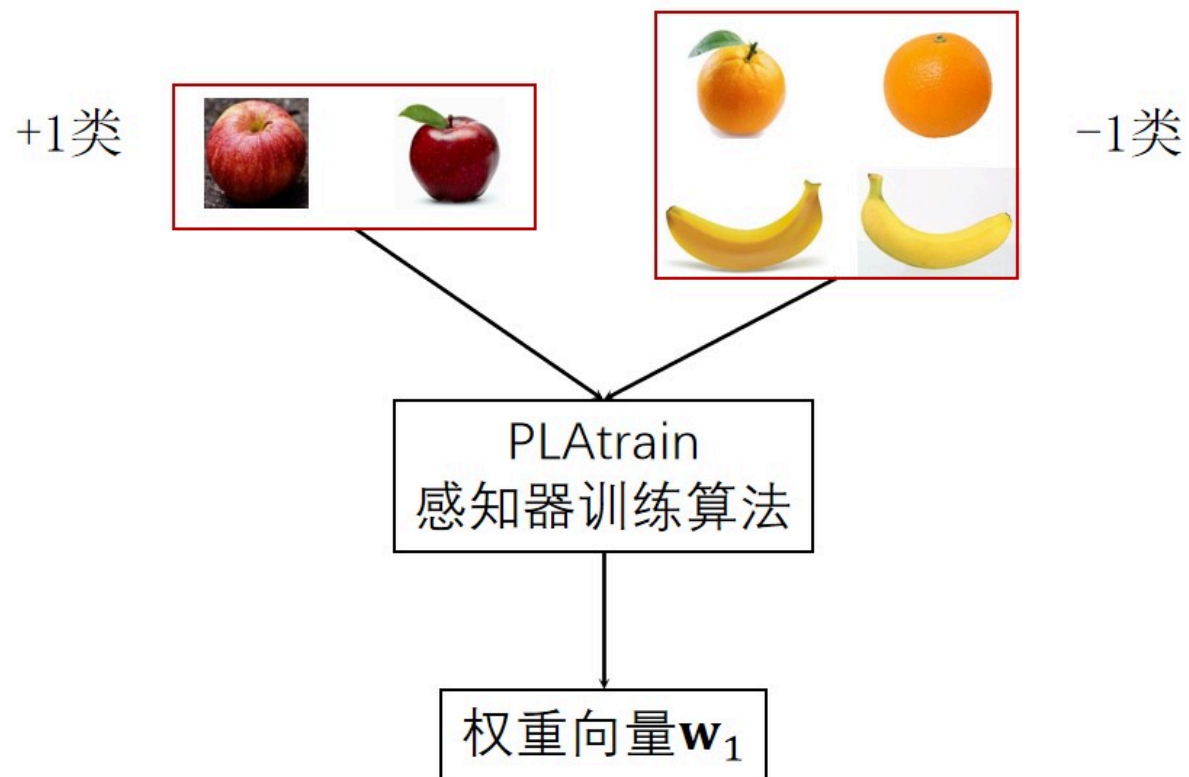
多类别感知器 (Multiclass Perceptron)

感知器本质上是一个二分类算法，如何将其推广至多类别的模式分类问题呢？

- ◆ 一对一 (One vs. One, OvO)
- ◆ 一对多 (One vs. Rest, OvR)

多类别感知器

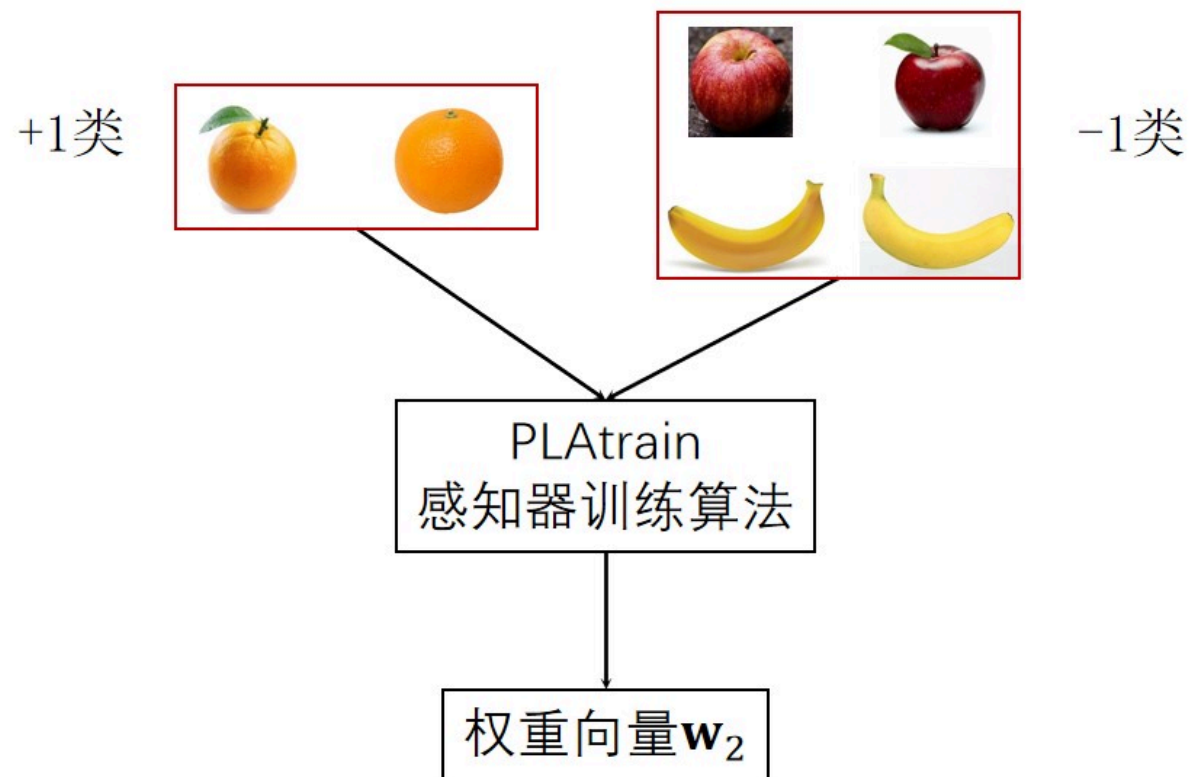
◆ 一对多 (One vs. Rest, OvR)



63

多类别感知器

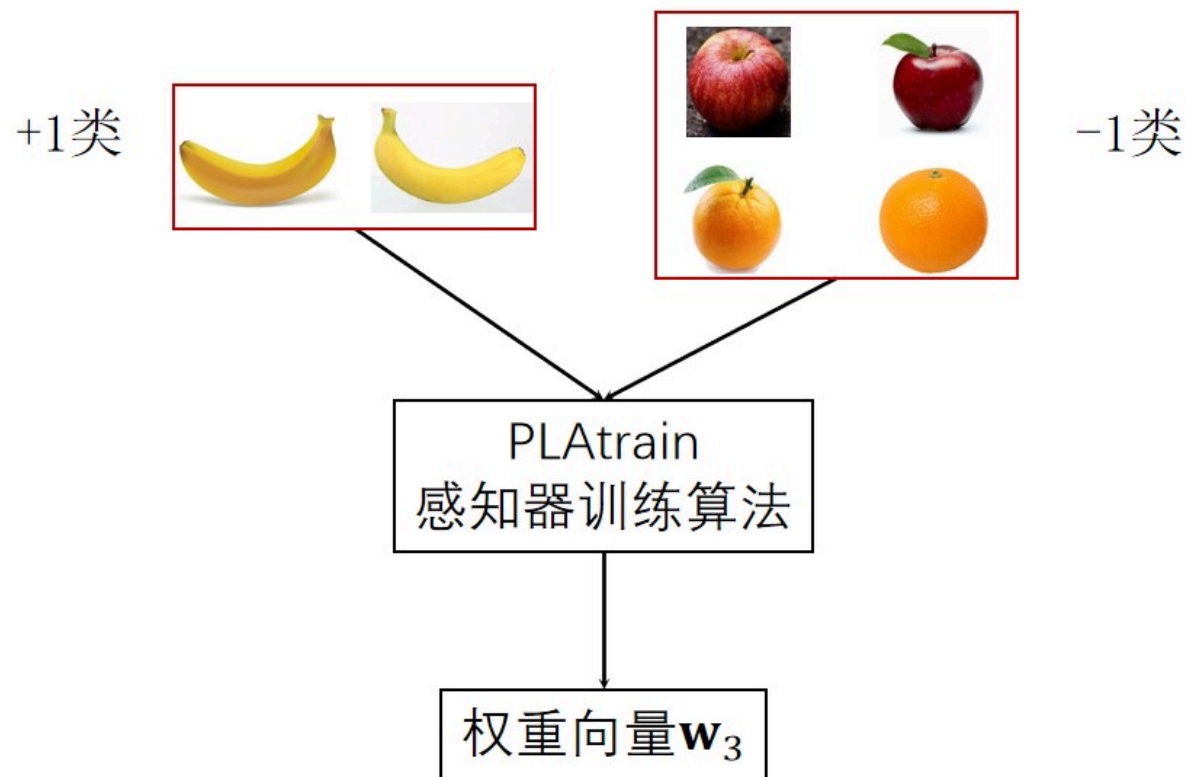
◆ 一对多 (One vs. Rest, OvR)



64

多类别感知器

◆ 一对多 (One vs. Rest, OvR)



65

多类别感知器

◆ 一对多 (One vs. Rest, OvR)

1. 那怎么分类呢?

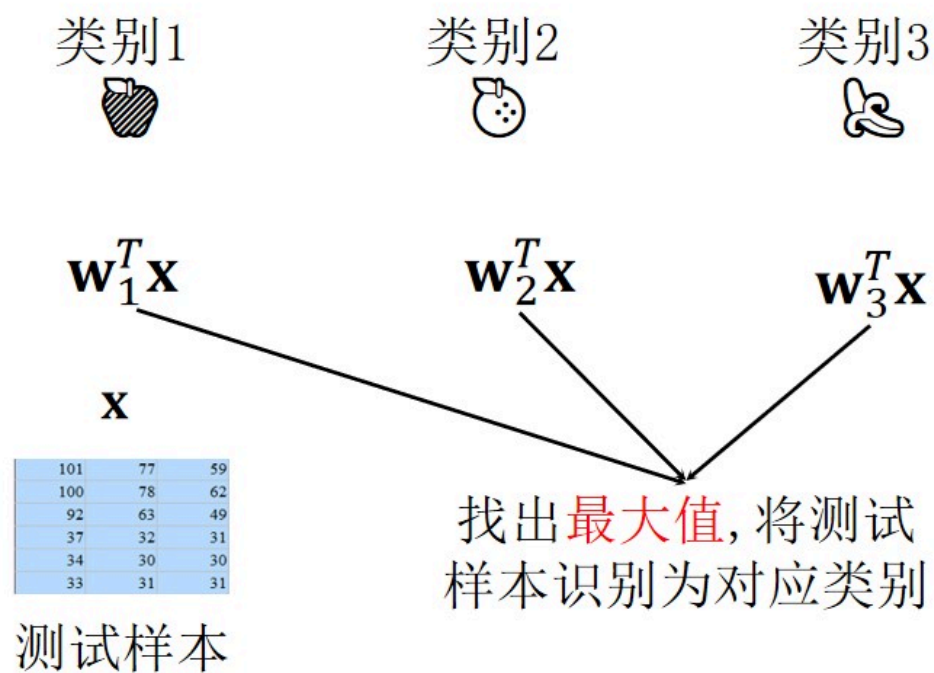
2. 还能像一对一的方法那样采用投票法吗?

3. 如果得出的分类结果为既是苹果又是香蕉怎么办?

66

多类别感知器

◆ 一对多 (One vs. Rest, OvR)



67

多类别感知器

◆ 一对多 (One vs. Rest, OvR)

类别1



类别2



类别3



$$w_1^T x$$

$$w_2^T x$$

$$w_3^T x$$

x



测试样本

识别为橙子

感知器

感知器学习算法优点：

- 简单、直观
- 测试速度快

感知器学习算法缺点：

- 只能处理线性可分的样本
- 训练速度较慢

代码案例

◆ 感知器使用代码 (采用Scikit-Learn)

案例：手写体数字识别

假设你要开发一个自动识别手写体数字的模型，如何利用Perceptron构建该模型。

1. 读取数据

首先，从sklearn中读取原始数据。

```
# 引入所需要的库文件
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, roc_auc_score
import matplotlib.pyplot as plt
from sklearn.linear_model import Perceptron

%config InlineBackend.figure_format = 'svg'
%matplotlib inline

# 加载数据集
X, y = load_digits(return_X_y=True)
```

70

代码案例

◆ 感知器使用代码 (采用Scikit-Learn)

案例：手写体数字识别

假设你要开发一个自动识别手写体数字的模型，如何利用Perceptron构建该模型。

sklearn.datasets.load_digits

```
sklearn.datasets.load_digits(*, n_class=10, return_X_y=False, as_frame=False)
```

[\[source\]](#)

Load and return the digits dataset (classification).

Each datapoint is a 8x8 image of a digit.

Classes	10
Samples per class	~180
Samples total	1797
Dimensionality	64
Features	integers 0-16

This is a copy of the test set of the UCI ML hand-written digits datasets
<https://archive.ics.uci.edu/ml/datasets/Optical+Recognition+of+Handwritten+Digits>

Read more in the User Guide.

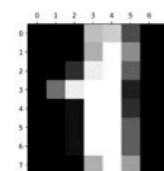
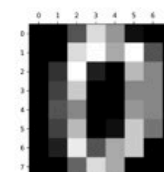
Parameters: **n_class** : int, default=10

The number of classes to return. Between 0 and 10.

return_X_y : bool, default=False

If True, returns (data, target) instead of a Bunch object. See below for more information about the data and target object.

New in version 0.18.



代码案例

◆ 感知器使用代码 (采用Scikit-Learn)

```
#查看数据的前2条  
print(X[0:2,:])  
print(y[0:2])
```

✓ 0.3s

```
[[ 0.  0.  5. 13.  9.  1.  0.  0.  0.  0. 13. 15. 10. 15.  5.  0.  0.  3.  
 15.  2.  0. 11.  8.  0.  0.  4. 12.  0.  0.  8.  8.  0.  0.  5.  8.  0.  
  0.  9.  8.  0.  0.  4. 11.  0.  1. 12.  7.  0.  0.  2. 14.  5. 10. 12.  
  0.  0.  0.  0.  6. 13. 10.  0.  0.  0.  0.]  
[ 0.  0.  0. 12. 13.  5.  0.  0.  0.  0.  0. 11. 16.  9.  0.  0.  0.  0.  
  3. 15. 16.  6.  0.  0.  0.  7. 15. 16. 16.  2.  0.  0.  0.  0.  1. 16.  
 16.  3.  0.  0.  0.  0.  1. 16. 16.  6.  0.  0.  0.  0.  1. 16. 16.  6.  
  0.  0.  0.  0.  0. 11. 16. 10.  0.  0.]]  
[0 1]
```

72

代码案例

◆ 感知器使用代码 (采用Scikit-Learn)

2. 训练模型

利用数据构建感知器模型

```
#利用经验数据构建感知器模型
X_train,X_test,y_train,y_test=train_test_split(X,y,train_size=0.7,test_size=0.3)
clf = Perceptron(tol=1e-3, random_state=0)
clf.fit(X_train, y_train)
```

3. 预测新数据

利用学习到的模型预测任意新输入

```
#利用学习到的模型预测任意新输入
y_pred=clf.predict(X_test)
```

代码案例

◆ 感知器使用代码 (采用Scikit-Learn)

4. 评价预测结果

计算预测结果的多种评价指标

```
#打印主要分类指标的文本报告
print('--- report ---')
print(classification_report(y_test,y_pred))
```

✓ 0.1s

```
--- report ---
              precision    recall  f1-score   support

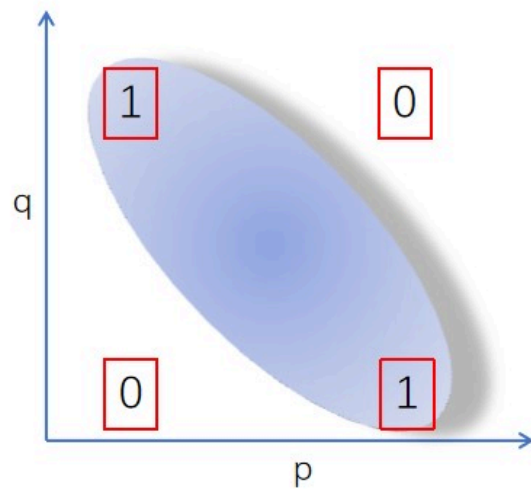
     0       0.98         1.00         0.99         59
     1       0.92         0.92         0.92         53
     2       0.92         1.00         0.96         49
     3       0.98         0.85         0.91         54
     4       0.97         1.00         0.98         56
     5       0.92         0.92         0.92         48
     6       0.97         1.00         0.98         57
     7       1.00         0.98         0.99         57
     8       0.89         0.88         0.88         56
     9       0.90         0.90         0.90         51

 accuracy                   0.95         540
```

74

多层感知器--算法原理

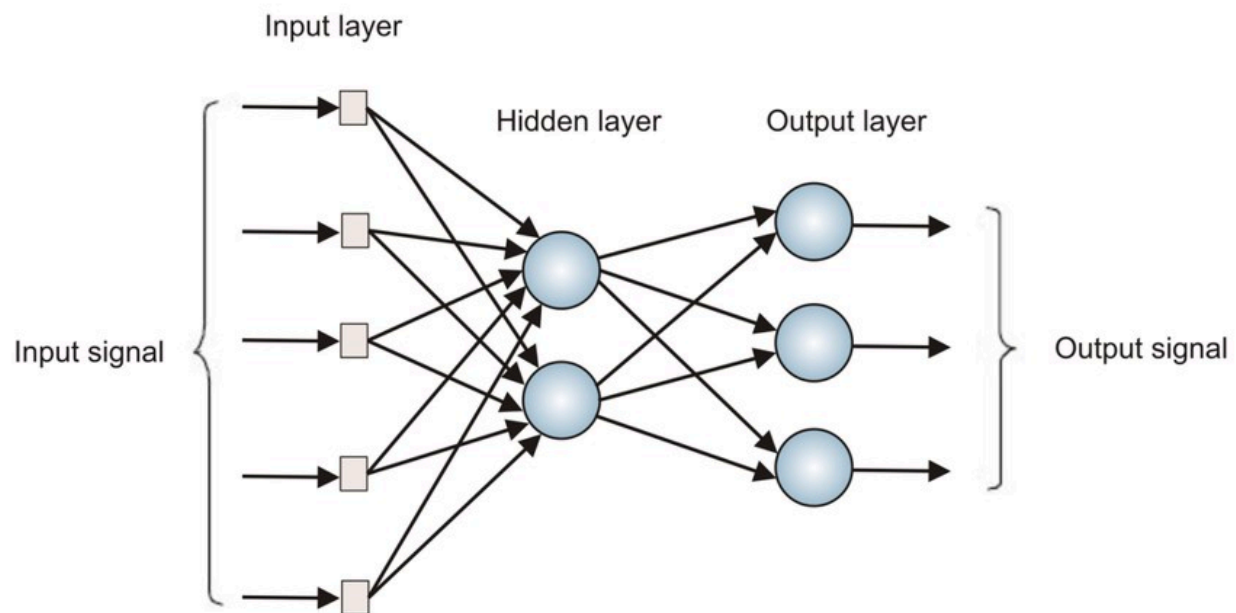
$$p \oplus q = p\bar{q} + \bar{p}q = (p + q)(\overline{pq})$$



Input		Output
0	0	0
0	1	1
1	0	1
1	1	0

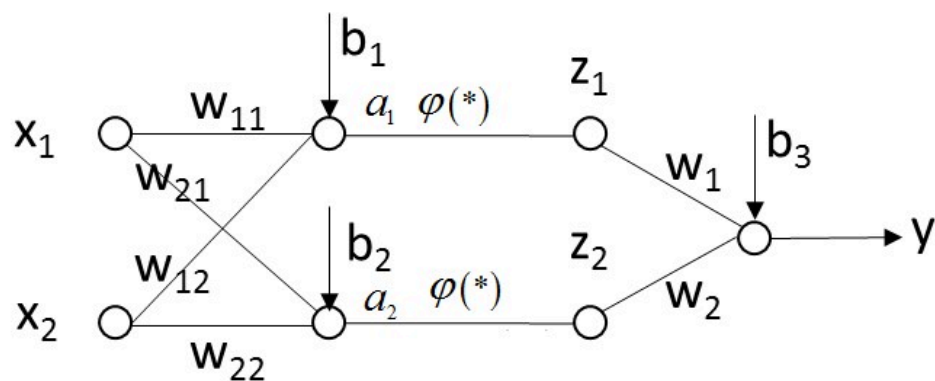
一根线不能把它们分开

多层感知机



多层感知机

两层神经网络例子：



$$a_1 = \omega_{11}x_1 + \omega_{12}x_2 + b_1$$

$$a_2 = \omega_{21}x_1 + \omega_{22}x_2 + b_2$$

$$z_1 = \varphi(a_1)$$

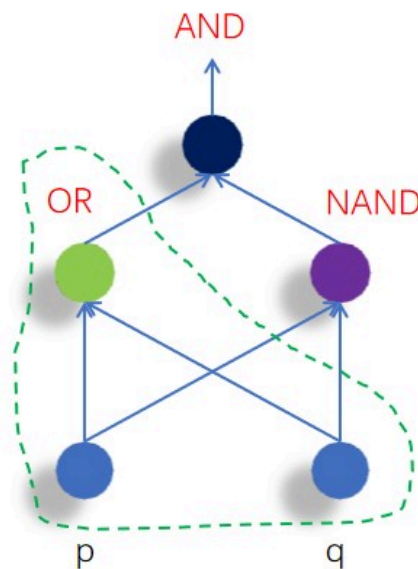
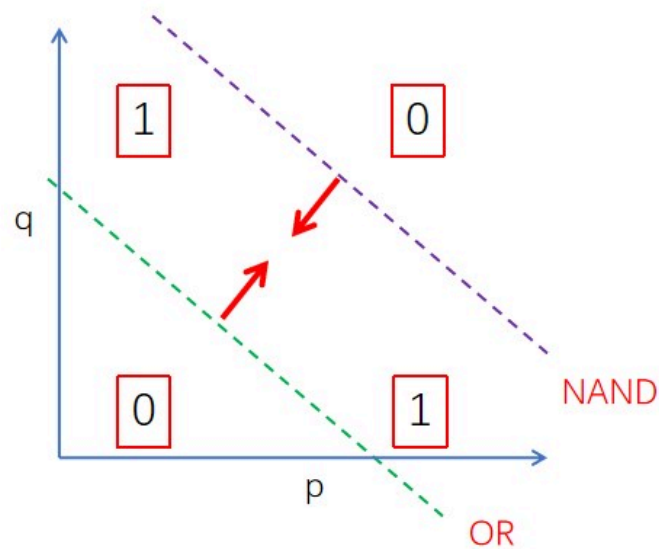
$$z_2 = \varphi(a_2)$$

$$y = \omega_1 z_1 + \omega_2 z_2 + b_3 \quad (\text{注意：其中 } \varphi(*) \text{ 为非线性函数})$$

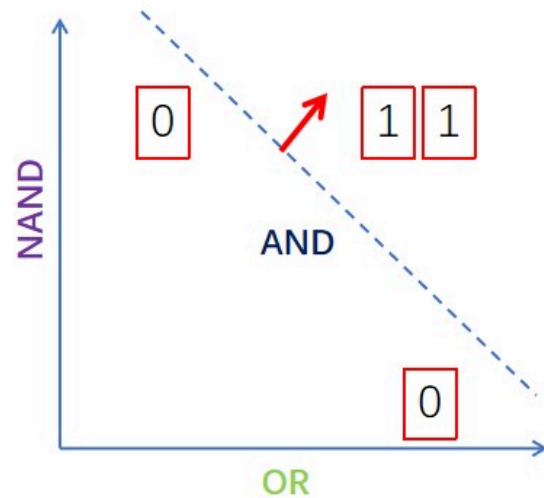
感知器--算法原理

异或问题 XOR

$$p \oplus q = \neg(p \wedge q) \wedge (p \vee q)$$

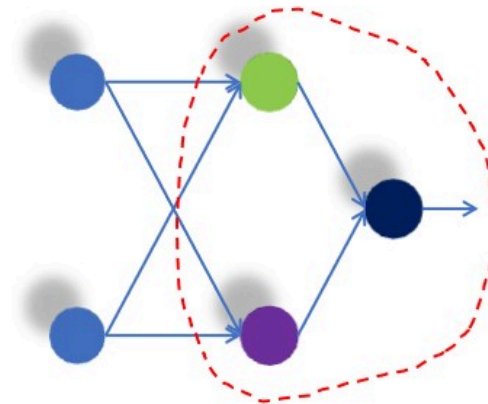


感知器--算法原理



p	q	OR	NAND	AND
0	0	0	1	0
0	1	1	1	1
1	0	1	1	1
1	1	1	0	0

Input Hidden Output

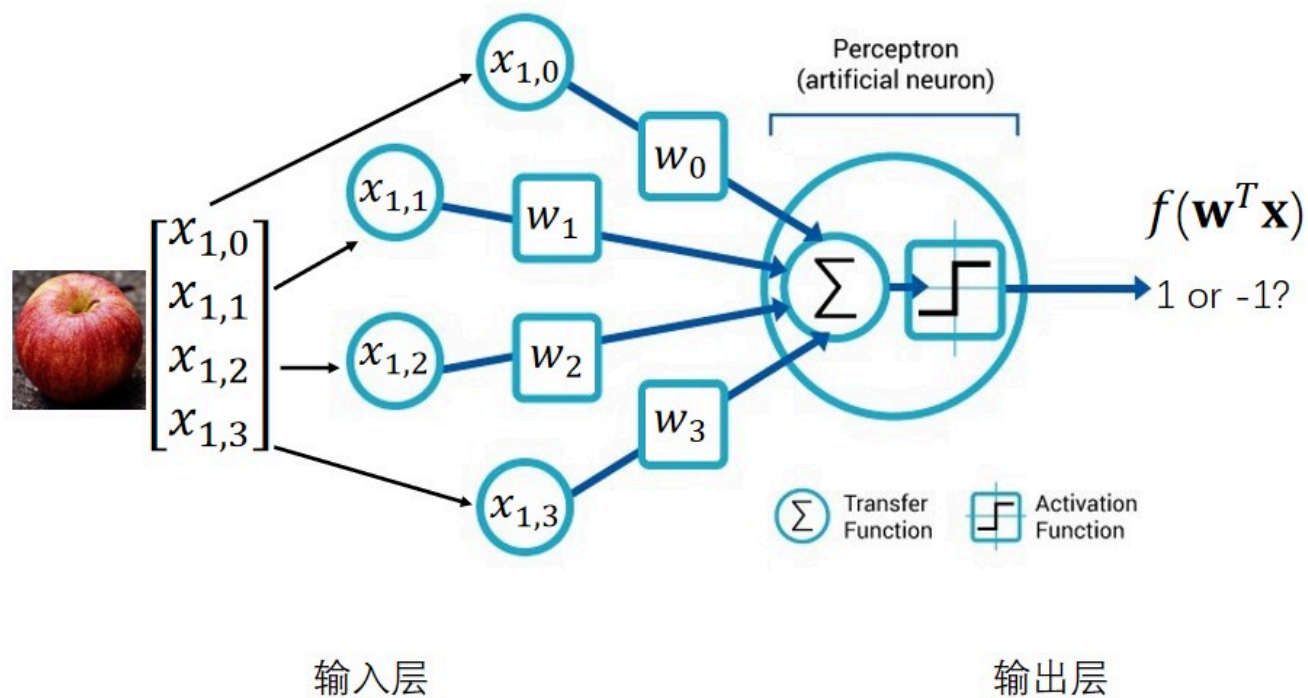




深度神经网络

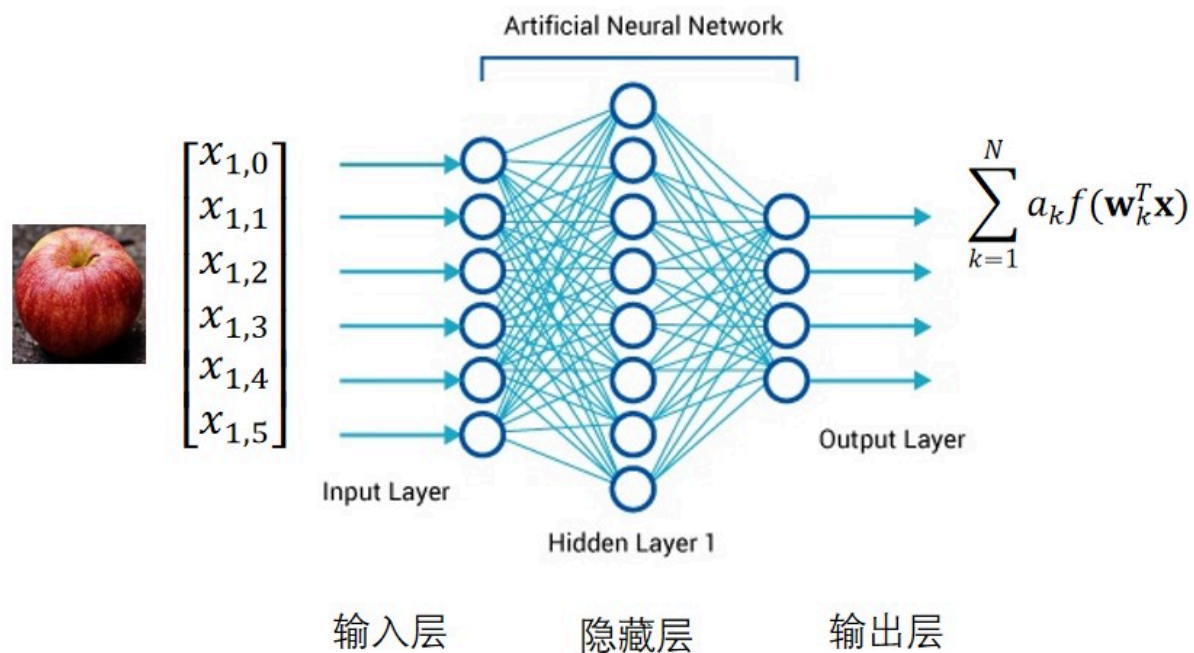
神经网络

单层感知器 (Single-layer Perceptron)



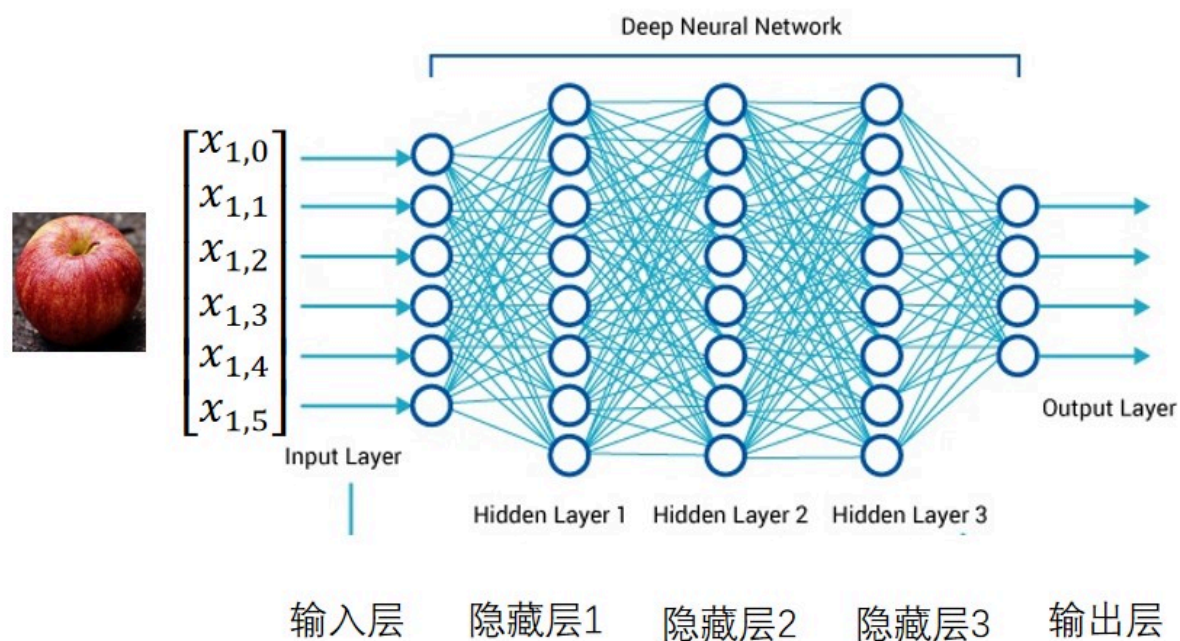
神经网络

浅层神经网络 (Shallow Neural Networks)

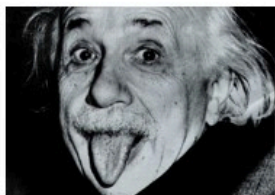


深度神经网络

深度神经网络 (Deep Neural Networks)



深度神经网络



Neural Nets

Perception

RNN (语音)

CNN (图像)

RBM

DBN

D-AE

AlexNet

GoogLeNet

McCulloch & Pitt 1943

Rosenblatt 1958

Grossberg 1973

Fukushima 1979

Hinton 1999

Hinton 2006

Vincent 2008

Alex 2012

Szegedy 2015

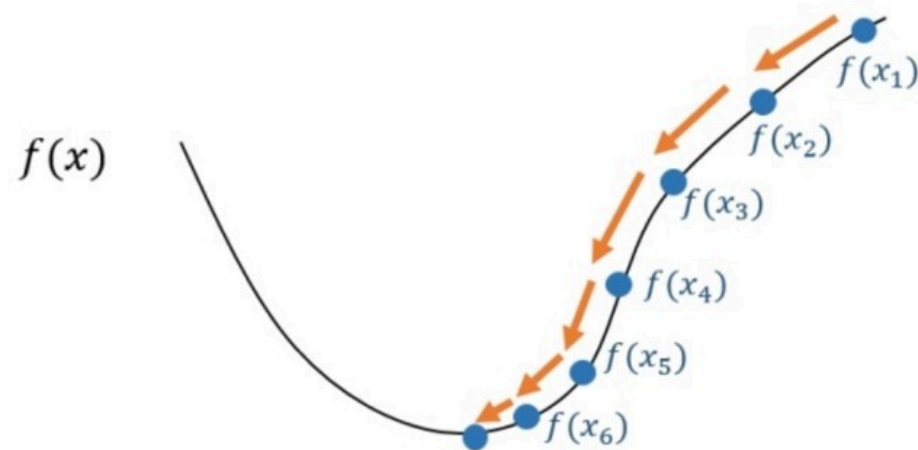
深度神经网络

如何训练神经网络中的权重参数呢？

85

深度神经网络

梯度下降 (Gradient Descent)

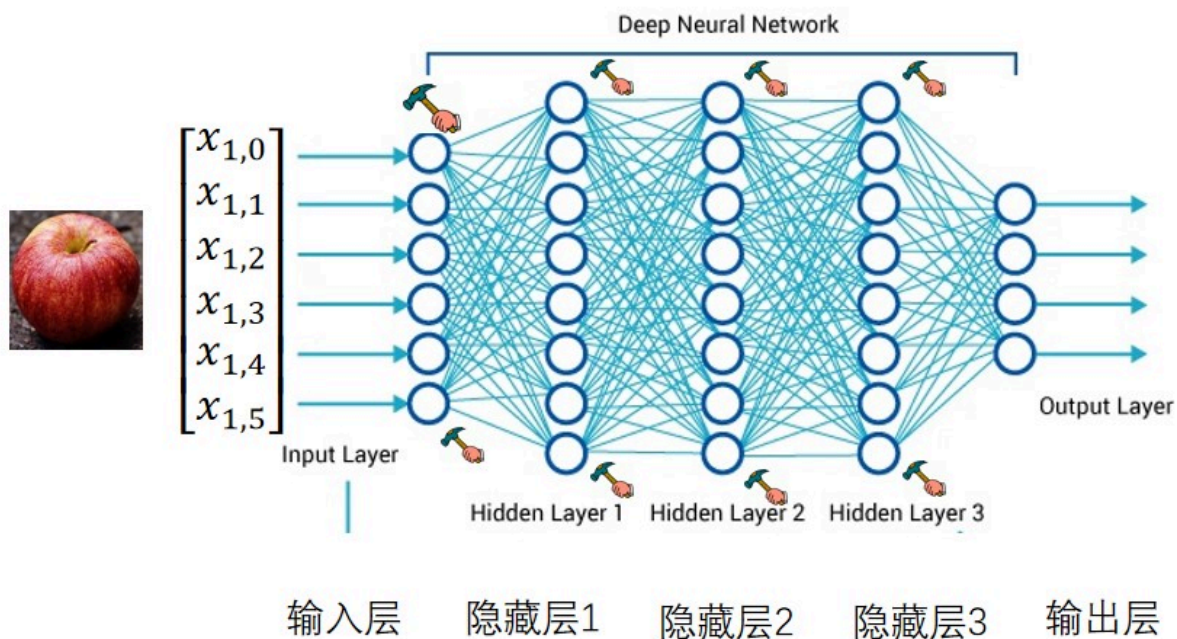


$$x_{t+1} = x_t - \eta \nabla f(x_t)$$

86

深度神经网络

深度神经网络 (Deep Neural Networks)



87

深度神经网络

批次梯度下降 (Batch Gradient Descent, BGD)

$$E(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n e_i(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n L(y_i, f(\mathbf{w}; \mathbf{x}_i))$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla E(\mathbf{w}_t)$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \frac{1}{n} \sum_{i=1}^n \nabla e_i(\mathbf{w}_t)$$

88

深度神经网络

随机梯度下降 (Stochastic Gradient Descent,SGD)

$$E(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n e_i(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n L(y_i, f(\mathbf{w}; \mathbf{x}_i))$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla e_i(\mathbf{w}_t)$$

每次迭代**随机**选择指标*i*

深度神经网络

小批量梯度下降 (Mini-batch Gradient Descent)

$$E(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n e_i(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n L(y_i, f(\mathbf{w}; \mathbf{x}_i))$$

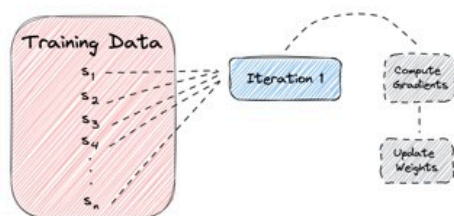
$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \frac{1}{|\mathcal{B}_k|} \sum_{i \in \mathcal{B}_k} e_i(\mathbf{w})$$

在训练前，将原始数据集分为 K 个批次， $\mathcal{B}_k$ 为第 k 个小批次数据的指标集合。

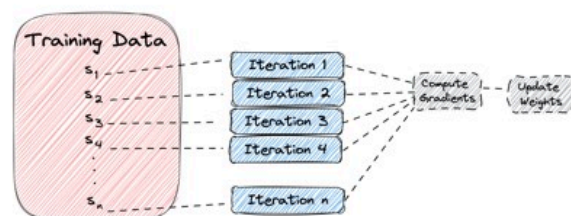
90

深度神经网络

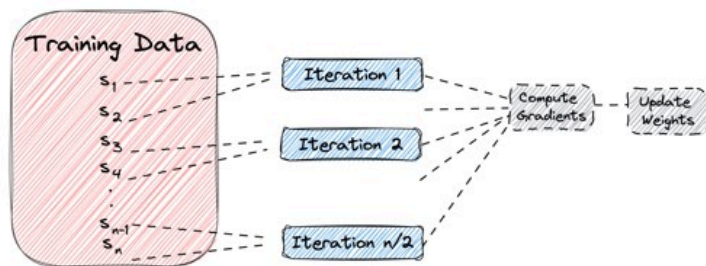
三种梯度下降方法对比



批量梯度下降



随机梯度下降



小批量梯度下降

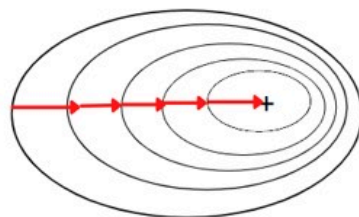
<https://www.baeldung.com/cs/epoch-vs-batch-vs-mini-batch>

91

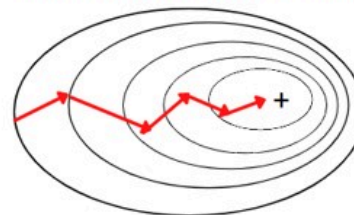
深度神经网络

三种梯度下降方法对比

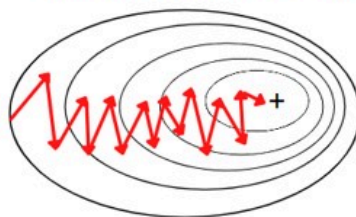
Batch Gradient Descent



Mini-Batch Gradient Descent



Stochastic Gradient Descent



<https://laptrinhx.com/understanding-optimization-algorithms-3818430905/>

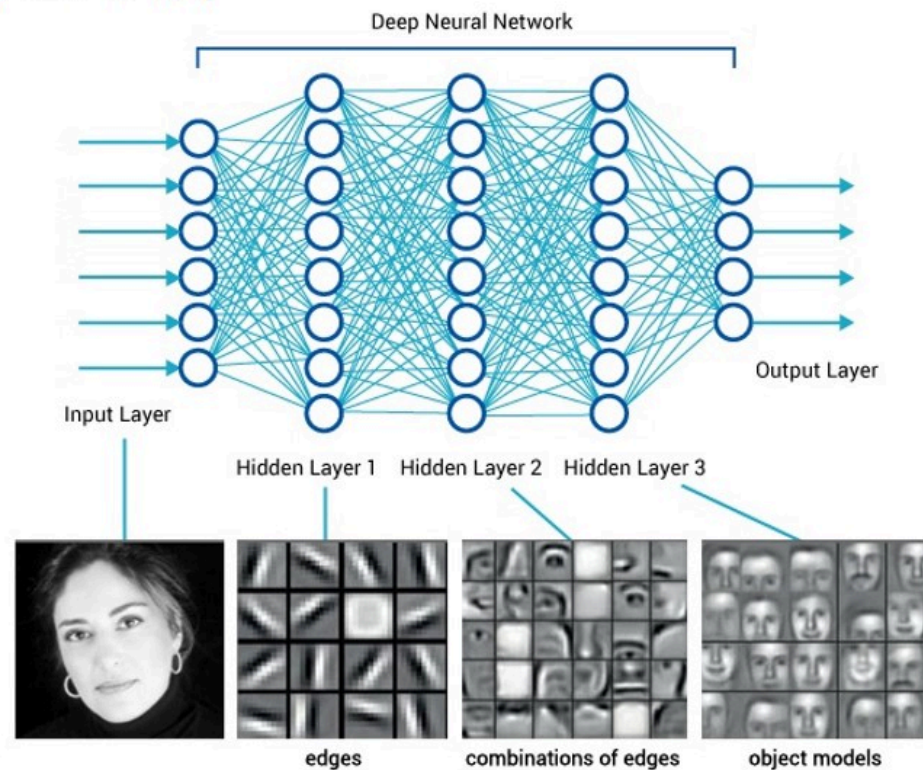
92



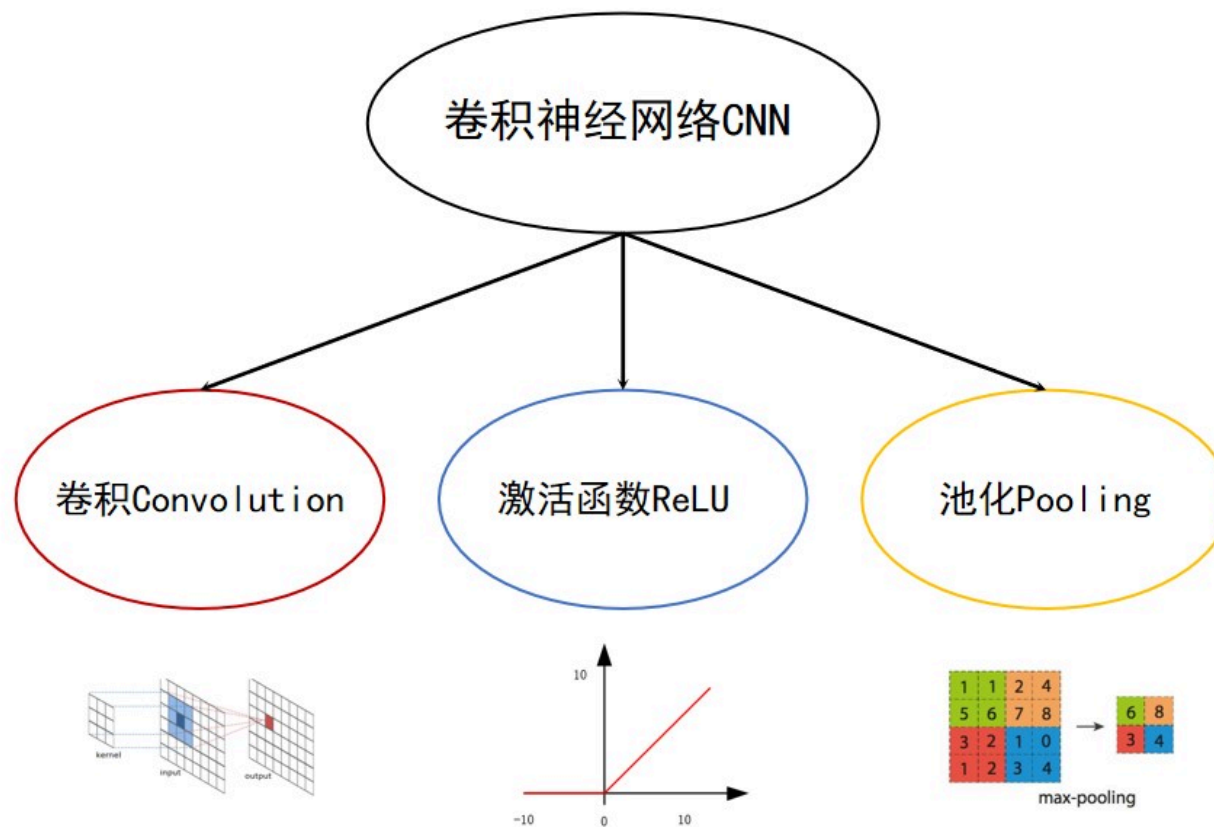
深度卷积神经网络

深度神经网络

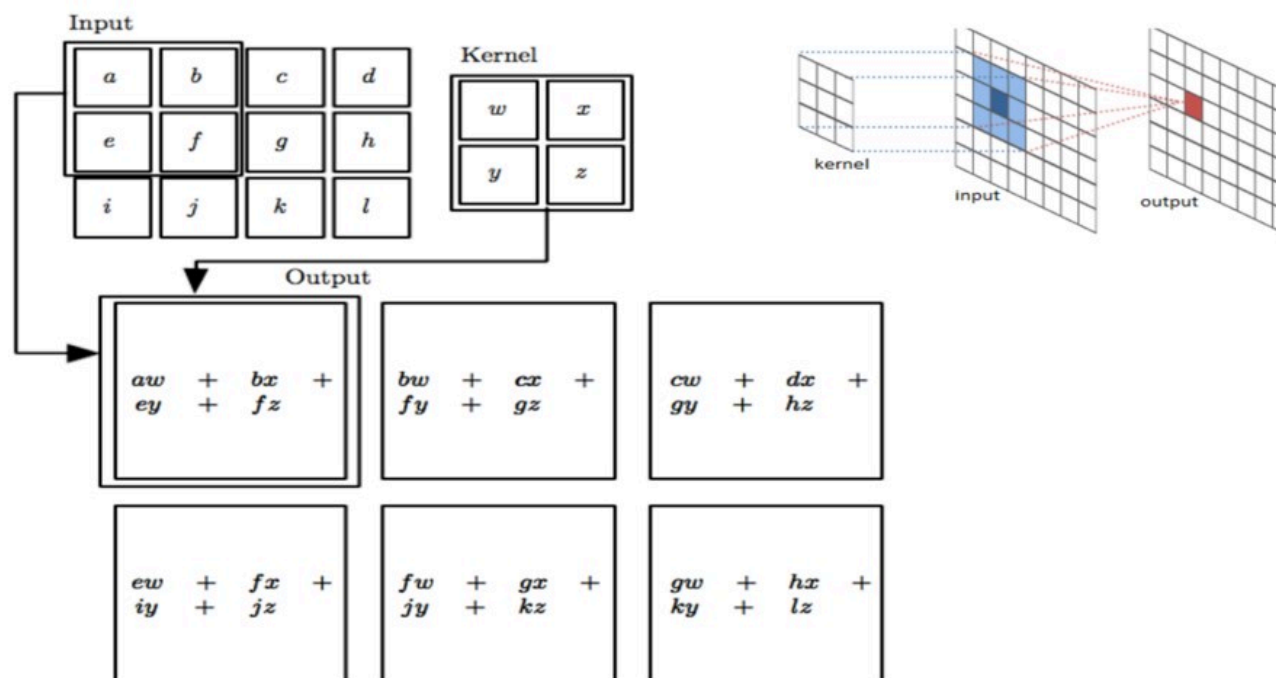
卷积神经网络 (Convolutional Neural Networks)



深度神经网络

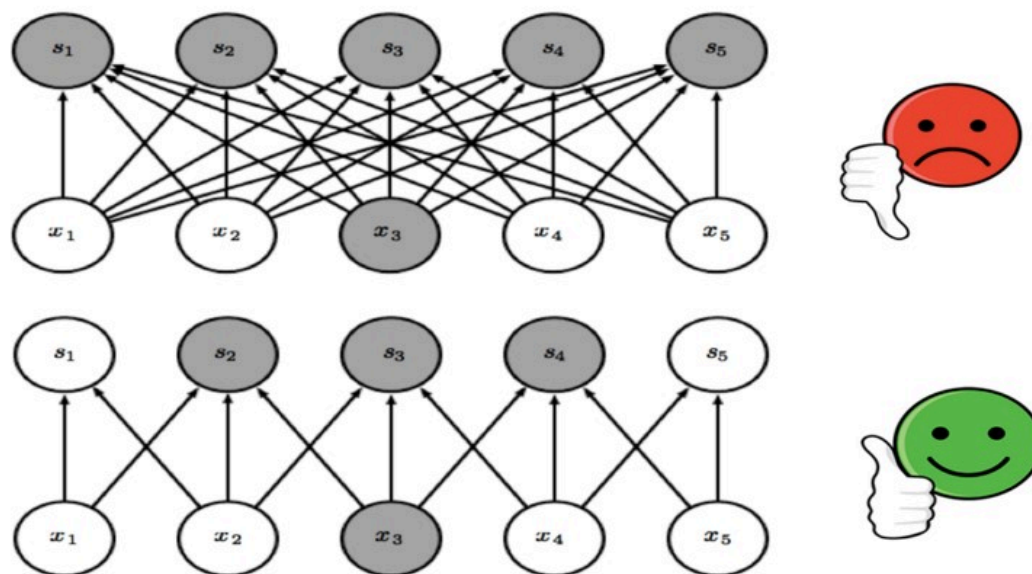


卷积Convolution



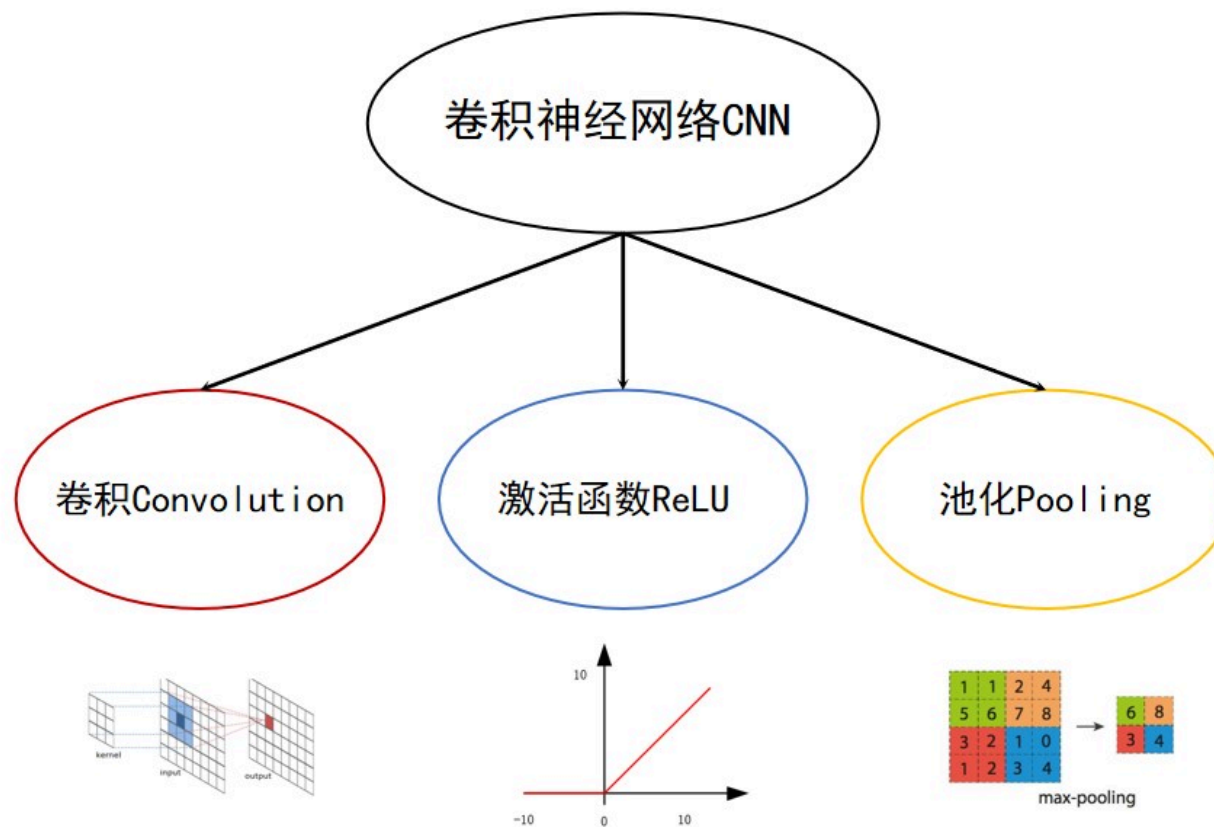
卷积Convolution

为什么需要卷积：稀疏连接 Sparse Connectivity



slide from: Fei-Fei Li & Andrej Karpathy & Justin Johnson

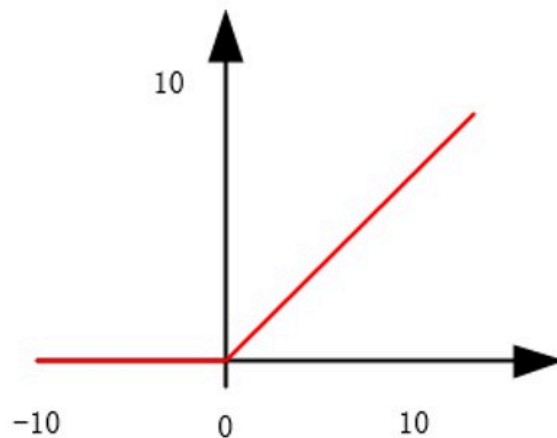
卷积神经网络



激活函数ReLU

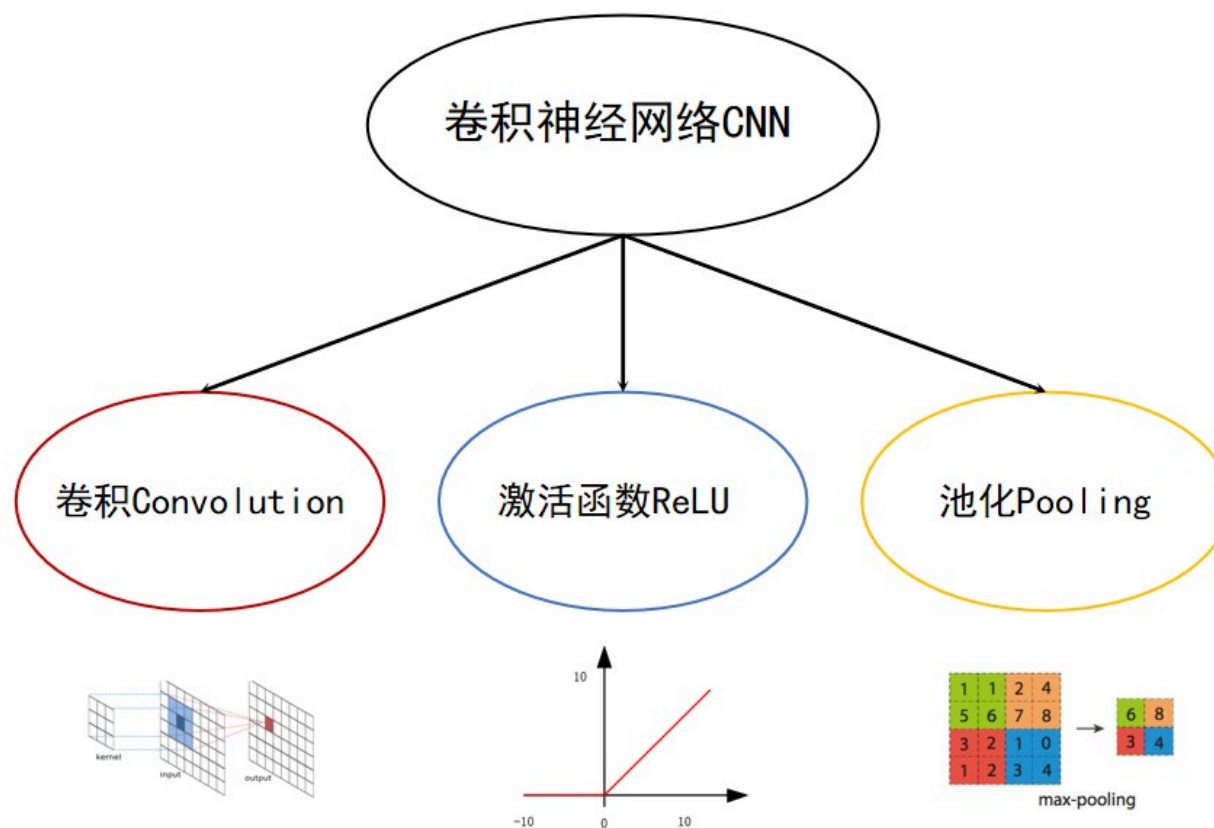
ReLU (Rectified Linear Unit 线性整流函数)

$$f(x) = \max(0, x)$$

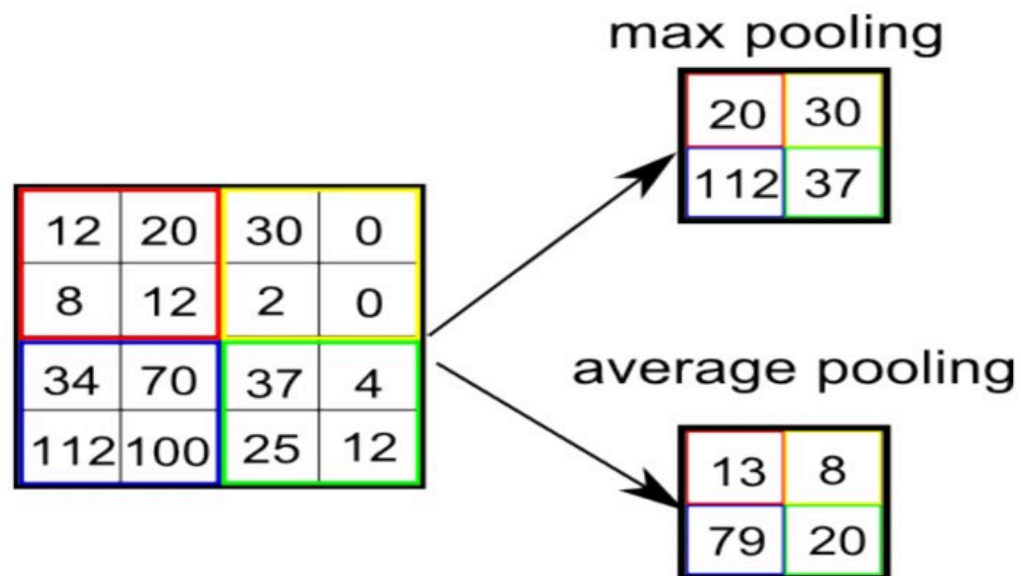


作用：引入非线性功能

卷积神经网络



池化Pooling

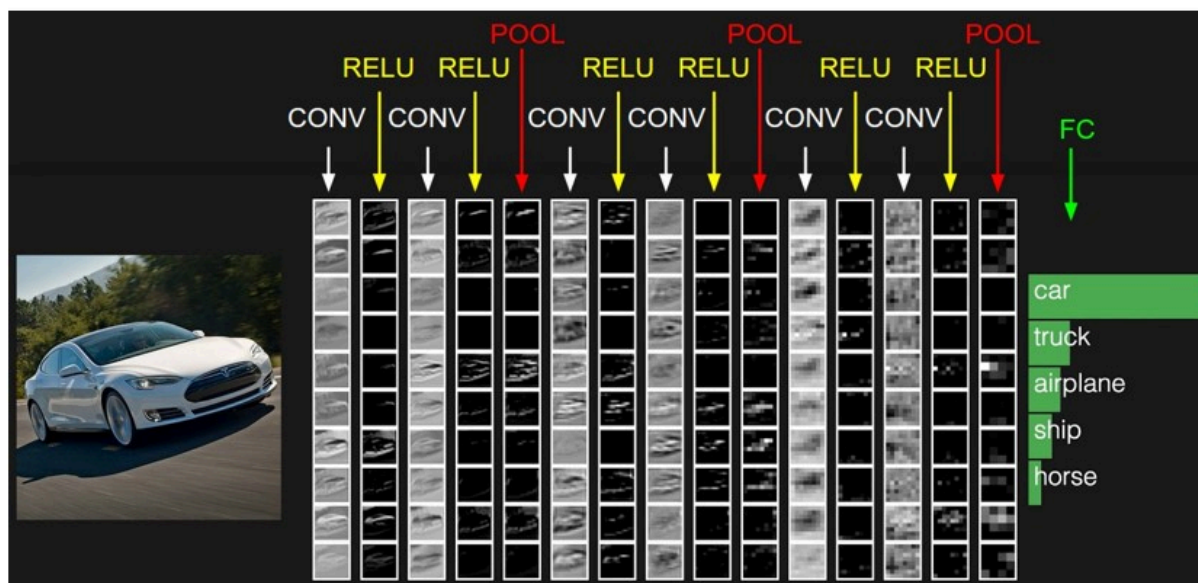


作用：1.减少特征维度，提升效率

2.对小幅度平移保持不变

卷积神经网络

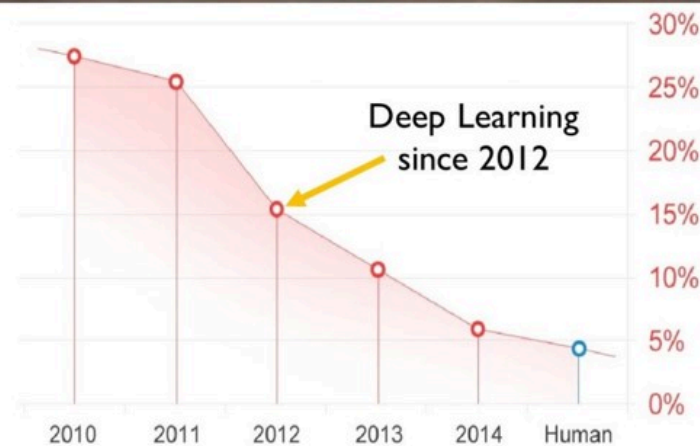
卷积神经网络 (Convolutional Neural Networks)

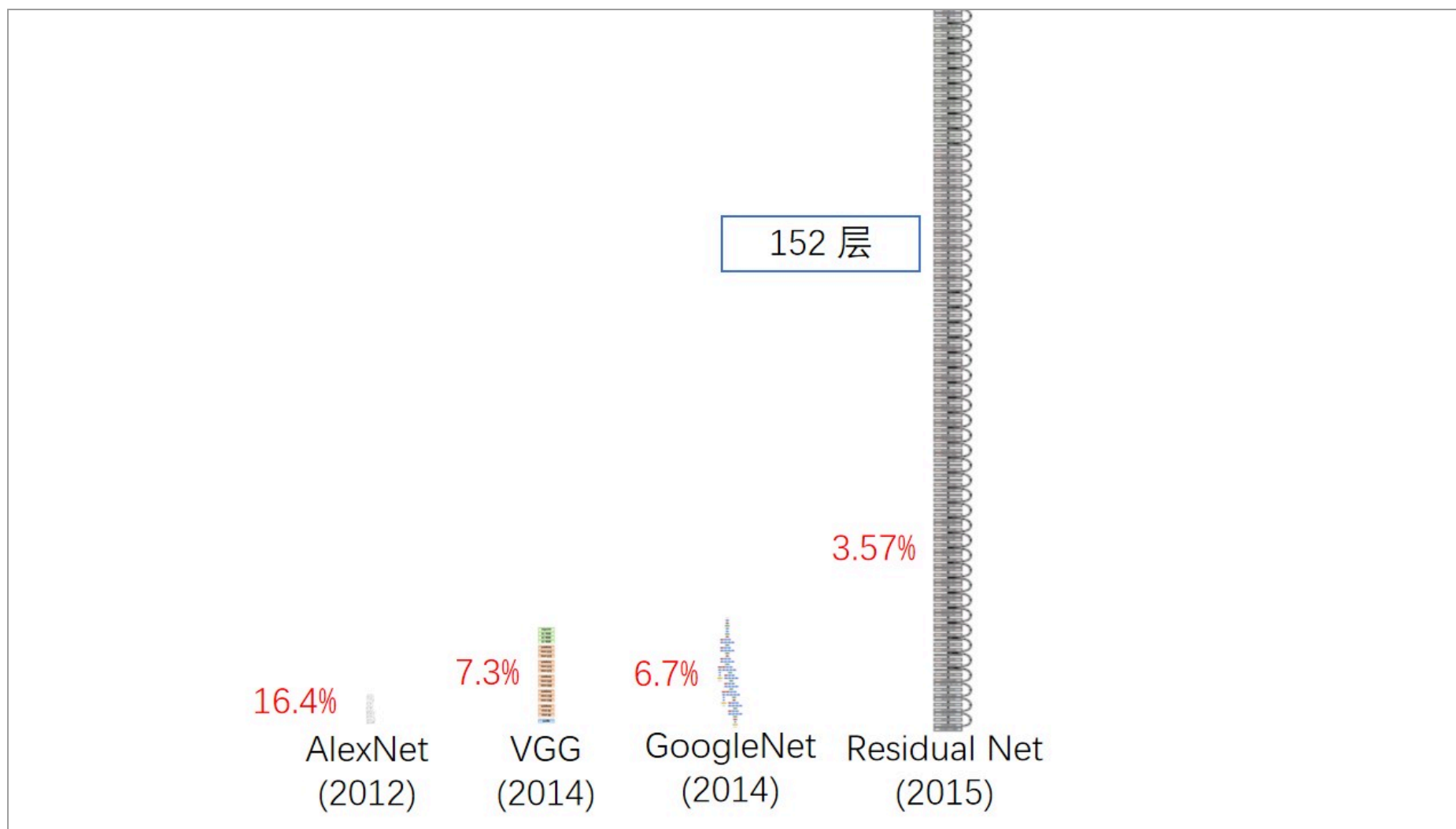


卷积神经网络

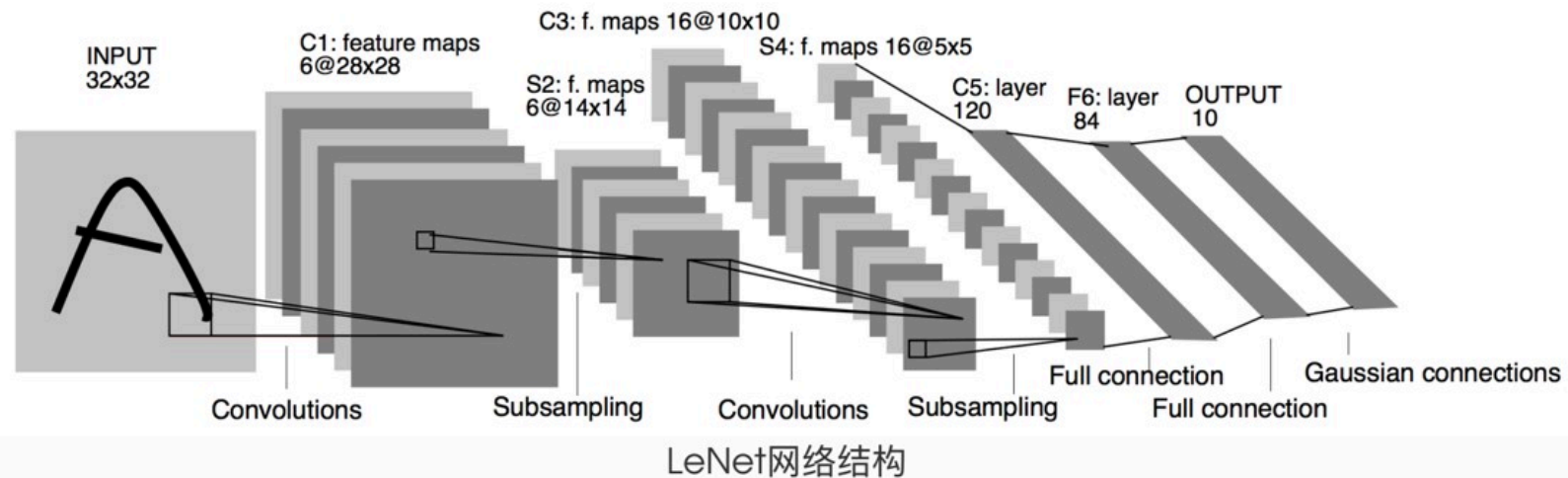


15,000,000 images
22,000 categories





LeNet

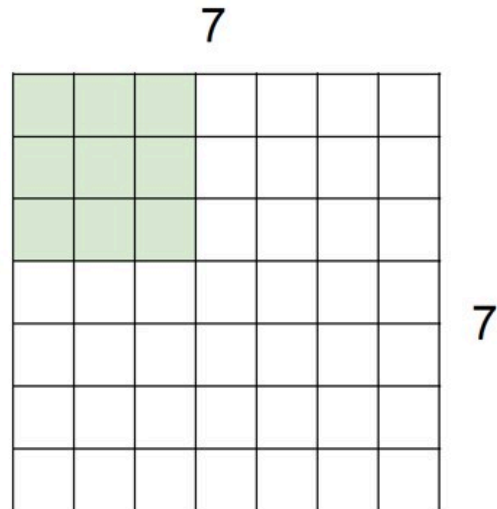


<https://zhuanlan.zhihu.com/p/162881214>

LeCun Y, Bottou L, Bengio Y, et al. Gradient-based learning applied to document recognition[J]. Proceedings of the IEEE, 1998, 86(11): 2278-2324.

LeNet

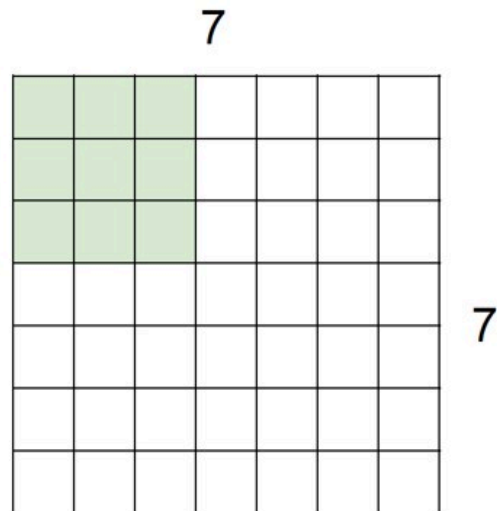
空间维数



- 输入: 7×7
- 卷积核: 3×3
- 步长: 1
- 输出: ?

LeNet

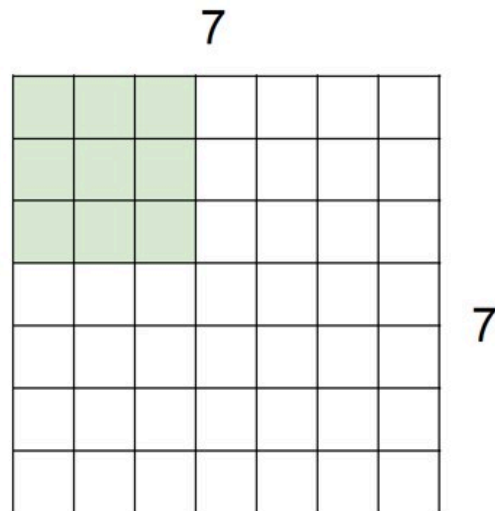
空间维数



- 输入: 7×7
- 卷积核: 3×3
- 步长: 1
- 输出: 5×5

LeNet

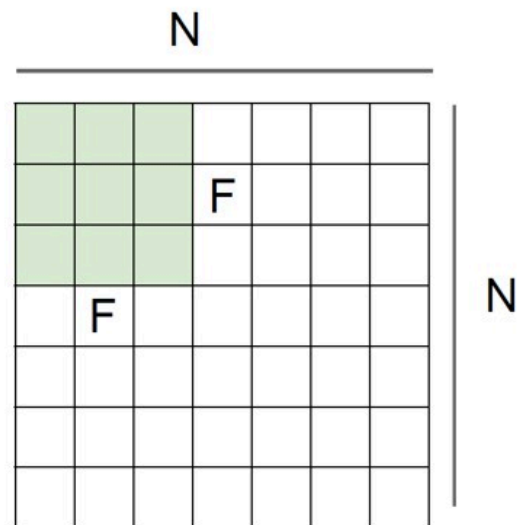
空间维数



- 输入: 7×7
- 卷积核: 3×3
- 步长: 1
- 输出: 5×5

LeNet

空间维数



➤ 输入: $N \times N$

➤ 卷积核: $F \times F$

➤ 步长: S

➤ 输出:

$$[(N-F)/S+1] \times [(N-F)/S+1]$$

LeNet

填充 (Padding)

0	0	0	0	0	0			
0								
0								
0								
0								

➤ 输入: $N \times N$

➤ 卷积核: $F \times F$

➤ 步长: S

➤ 填充: P

➤ 输出:

$$[(N-F+2P)/S+1] \times [(N-F+2P)/S+1]$$

111

LeNet

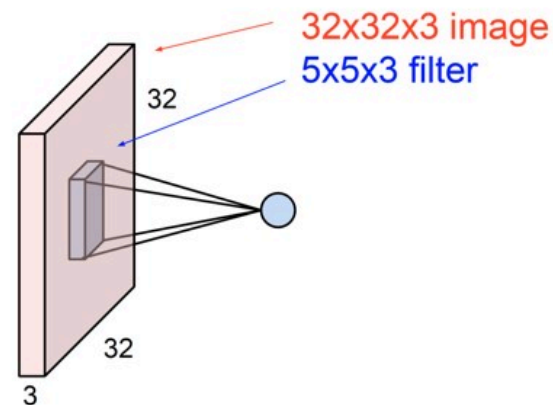
- 输入: $32 \times 32 \times 3$
- 10个卷积核: $5 \times 5 \times 3$
- 步长: 1
- 填充: 2
- 输出: ?

112

LeNet

➤ 输出维数

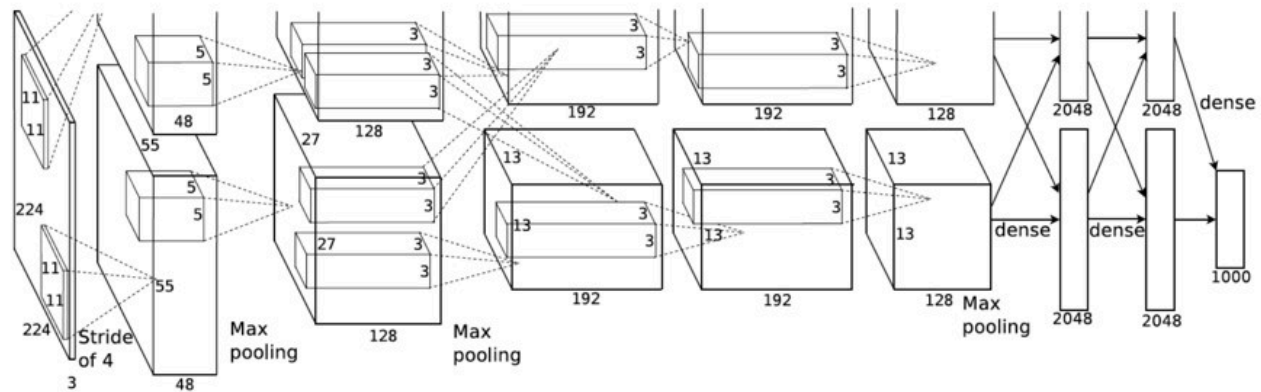
- 输入: $32 \times 32 \times 3$
- 10个卷积核: $5 \times 5 \times 3$
- 步长: 1
- 填充: 2
- 输出:



$$(N-F+2P)/S+1=(32-5+2 \times 2)/1+1=32$$

$32 \times 32 \times 10$

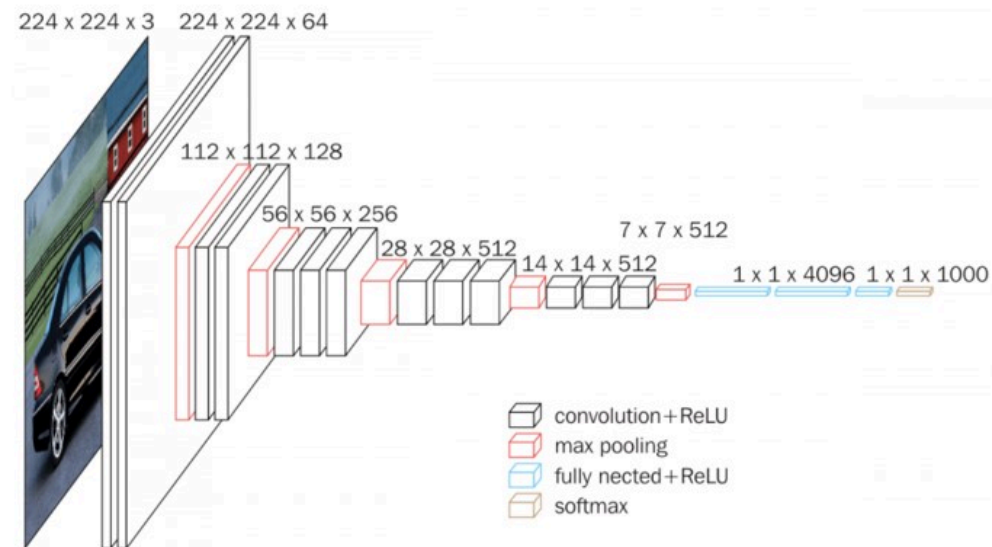
AlexNet



AlexNet网络结构 (2012)

A. Krizhevsky, I. Sutskever, and G. Hinton, "Imagenet classification with deep convolutional neural networks," NIPS, pp. 1097-1105, 2012.

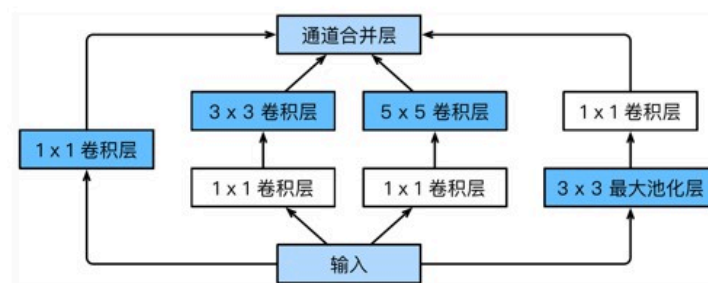
VGG



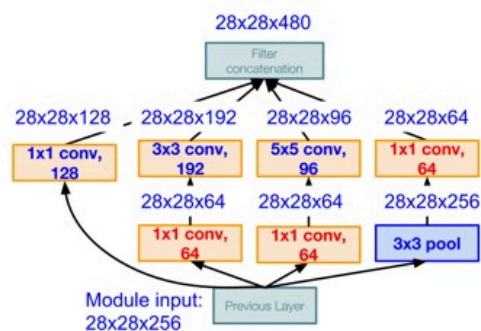
VGG 16 网络结构 (2014)

Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556.

GoogleNet

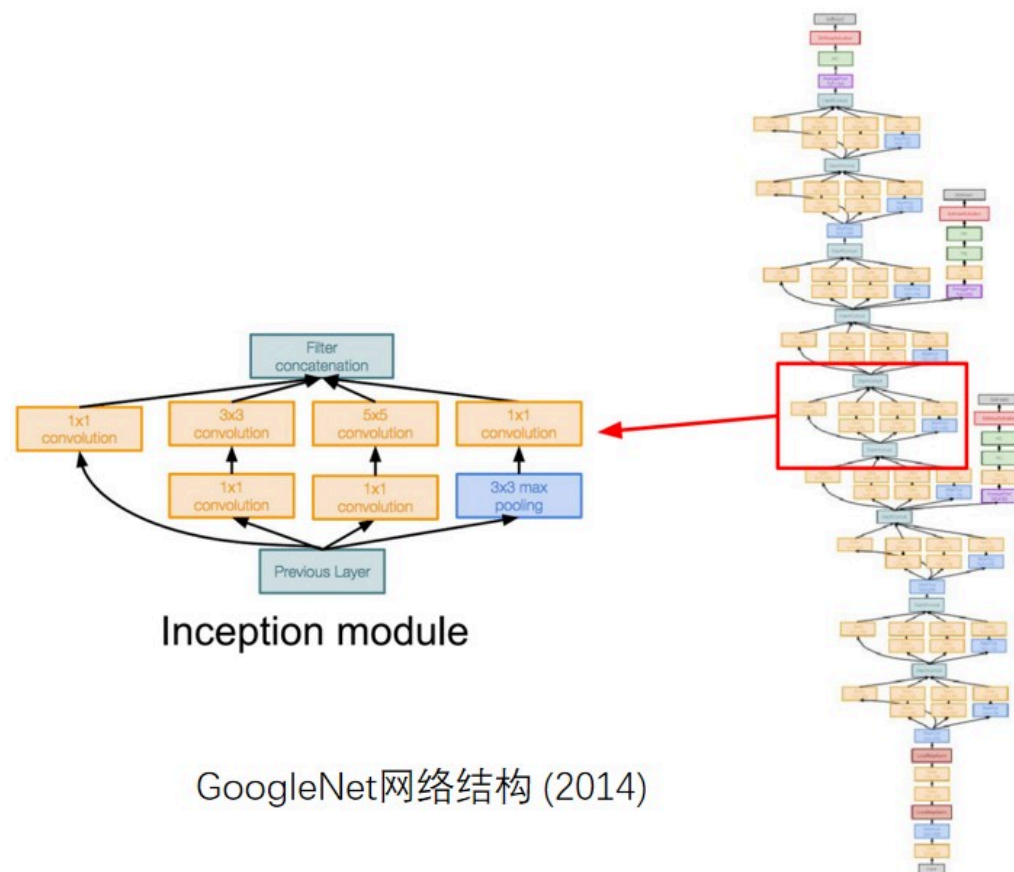


Inception块的结构

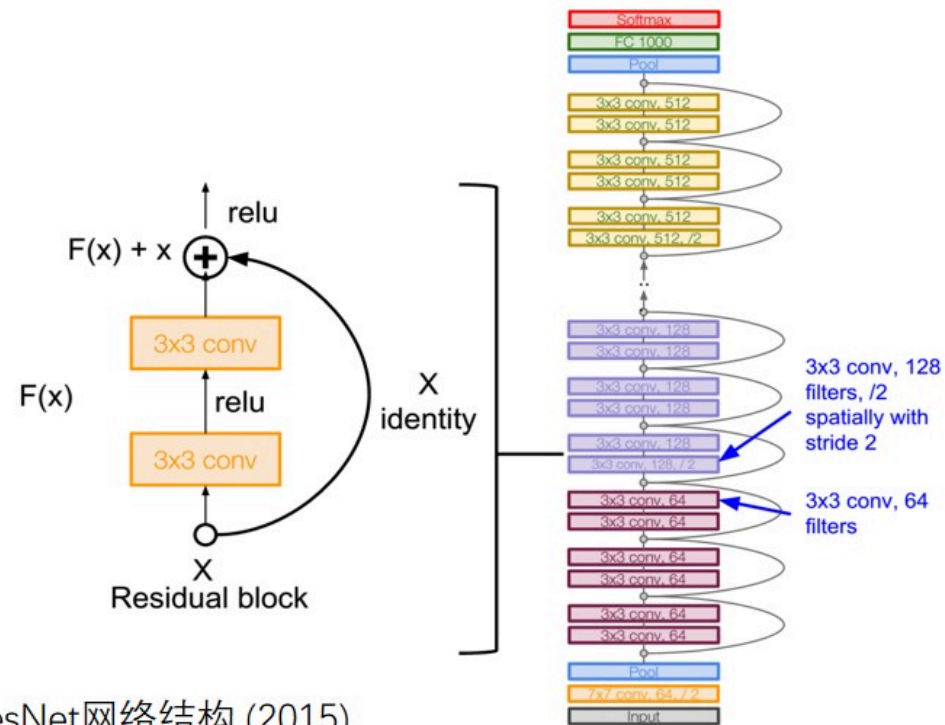


Szegedy C, Liu W, Jia Y, et al. Going deeper with convolutions, CVPR, 2015: 1-9.

GoogleNet



ResNet



ResNet网络结构 (2015)

He K, Zhang X, Ren S, et al. Deep residual learning for image recognition, CVPR, 2016: 770-778.

深度神经网络

优点：

- 效果好
- 特征学习能力强

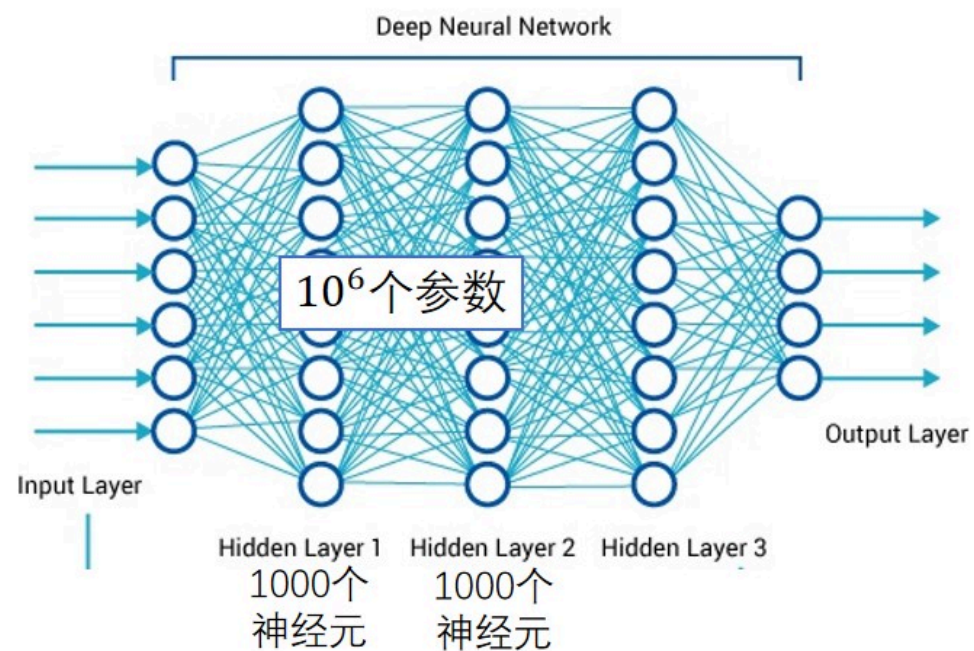
缺点：

- 需要**巨量**有标记数据
- 需要大量计算资源

这也解释了为什么DNN虽然很早发明，但现在才取得成功。

深度神经网络

- 需要巨量有标记数据,大量计算资源



为了准确估计这些权重参数, 需要巨量标记数据。

深度学习

□ 理解深度学习

“特征工程” VS “特征学习”或者“表示学习”

- 特征工程由人类专家根据现实任务来设计，特征提取与识别是单独的两个阶段；



手工设计
特征

分类识别

- 特征学习通过深度学习技术自动产生有益于分类的特征，是一个端到端的学习框架。



学习生成
特征

分类识别

深度学习

□ 深度学习常用软件包

● CAFFE

The Berkeley Vision and learning Center (BVLC)

<http://caffe.berkeleyvision.org/>

● MatConvNet

The Oxford Visual Geometry Group (VGG)

<http://www.vlfeat.org/matconvnet/>

● Torch

<http://torch.cn/>

□ 神经网络基础教材

<http://hagan.okstate.edu/>

阅读材料

□ 主流学术期刊

- Neural Computation
- Neural Networks
- IEEE Transactions on Neural Networks and Learning Systems

□ 国际会议

- 国际神经信息处理系统会议 (NIPS)
- 国际神经网络联合会议 (IJCNN)

□ 区域性国际会议

- 欧洲神经网络会议 (ICANN)
- 亚太神经网络会议 (ICONIP)

谢 谢!